LeetCode Practice

43 Medium: Stack

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

43
MEDIUM

Patterns (in order)

1. Stack (43)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

43 questions | tap a title to jump to it

Medium questions (43)	Go >
Stack (43)	**Go >**
M 1. Simplify Path	LC 71
M 2. Flatten Binary Tree to Linked List	LC 114
M 3. Reorder List	LC 143
M 4. Evaluate Reverse Polish Notation	LC 150
M 5. Min Stack	LC 155
M 6. Binary Search Tree Iterator	LC 173
M 7. Basic Calculator II	LC 227
M 8. Verify Preorder Sequence in Binary Search Tree	LC 255
M 9. Remove Duplicate Letters	LC 316
M 10. Decode String	LC 394
M 11. Remove K Digits	LC 402
M 12. Convert Binary Search Tree to Sorted Doubly Linked List	LC 426
M 13. Ternary Expression Parser	LC 439
M 14. Add Two Numbers II	LC 445
M 15. 132 Pattern	LC 456
M 16. Find Permutation	LC 484
M 17. Next Greater Element II	LC 503
M 18. Shortest Unsorted Continuous Subarray	LC 581
M 19. Exclusive Time of Functions	LC 636
M 20. Maximum Binary Tree	LC 654
M 21. Valid Parenthesis String	LC 678
M 22. Asteroid Collision	LC 735
M 23. Daily Temperatures	LC 739
M 24. Max Chunks To Make Sorted	LC 769
M 25. Car Fleet	LC 853
M 26. Online Stock Span	LC 901
M 27. Sum of Subarray Minimums	LC 907
M 28. Maximum Sum Circular Subarray	LC 918
M 29. Minimum Add to Make Parentheses Valid	LC 921
M 30. Validate Stack Sequences	LC 946
M 31. Maximum Width Ramp	LC 962
M 32. Construct Binary Search Tree from Preorder Traversal	LC 1008
M 33. Next Greater Node In Linked List	LC 1019
M 34. Two Sum BSTs	LC 1214
M 35. Minimum Remove to Make Valid Parentheses	LC 1249
M 36. Design Browser History	LC 1472
M 37. Minimum Deletions to Make String Balanced	LC 1653
M 38. Find the Most Competitive Subsequence	LC 1673
M 39. Maximum Score From Removing Substrings	LC 1717
M 40. Buildings With an Ocean View	LC 1762
M 41. Sum of Subarray Ranges	LC 2104
M 42. Removing Stars From a String	LC 2390
M 43. Count Partitions With Max-Min Difference at Most K	LC 3578

Medium questions (43)

MEDIUM | TOPIC 1 OF 1

Stack

43 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Simplify Path LC 71
2. Flatten Binary Tree to Linked List LC 114
3. Reorder List LC 143
4. Evaluate Reverse Polish Notation LC 150
5. Min Stack LC 155
6. Binary Search Tree Iterator LC 173
7. Basic Calculator II LC 227
8. Verify Preorder Sequence in Binary Search Tree LC 255
9. Remove Duplicate Letters LC 316
10. Decode String LC 394
11. Remove K Digits LC 402
12. Convert Binary Search Tree to Sorted Doubly Linked List LC 426
13. Ternary Expression Parser LC 439
14. Add Two Numbers II LC 445
15. 132 Pattern LC 456
16. Find Permutation LC 484
17. Next Greater Element II LC 503
18. Shortest Unsorted Continuous Subarray LC 581
19. Exclusive Time of Functions LC 636
20. Maximum Binary Tree LC 654
21. Valid Parenthesis String LC 678
22. Asteroid Collision LC 735
23. Daily Temperatures LC 739
24. Max Chunks To Make Sorted LC 769
25. Car Fleet LC 853
26. Online Stock Span LC 901
27. Sum of Subarray Minimums LC 907
28. Maximum Sum Circular Subarray LC 918
29. Minimum Add to Make Parentheses Valid LC 921
30. Validate Stack Sequences LC 946
31. Maximum Width Ramp LC 962
32. Construct Binary Search Tree from Preorder Traversal LC 1008
33. Next Greater Node In Linked List LC 1019
34. Two Sum BSTs LC 1214
35. Minimum Remove to Make Valid Parentheses LC 1249
36. Design Browser History LC 1472
37. Minimum Deletions to Make String Balanced LC 1653
38. Find the Most Competitive Subsequence LC 1673
39. Maximum Score From Removing Substrings LC 1717
40. Buildings With an Ocean View LC 1762
41. Sum of Subarray Ranges LC 2104
42. Removing Stars From a String LC 2390
43. Count Partitions With Max-Min Difference at Most K LC 3578

MEDIUM | Stack | LeetCode #71 | 1 of 43 | 1/43

Simplify Path

QUESTION

You are given an absolute path for a Unix-style file system, which always begins with a slash '/'. Your task is to transform this absolute path into its simplified canonical path.

The rules of a Unix-style file system are as follows:

CONSTRAINTS

- A single period '.' represents the current directory.
- A double period '..' represents the previous/parent directory.
- Multiple consecutive slashes such as '//' and '///' are treated as a single slash '/'.
- Any sequence of periods that does not match the rules above should be treated as a valid directory or file name. For example, '...' and '....' are valid directory or file names.

The simplified canonical path should follow these rules:

- The path must start with a single slash '/'.
- Directories within the path must be separated by exactly one slash '/'.
- The path must not end with a slash '/', unless it is the root directory.
- The path must not have any single or double periods ('.' and '..') used to denote current or parent directories.

Return the simplified canonical path.

Example 1

Input	`path = "/home/"`
Output	`"/home"`
Explanation	*The trailing slash should be removed.*

Example 2

Input	`path = "/home//foo/"`
Output	`"/home/foo"`
Explanation	*Multiple consecutive slashes are replaced by a single one.*

Example 3

Input	`path = "/home/user/Documents/../Pictures"`
Output	`"/home/user/Pictures"`
Explanation	*A double period ".." refers to the directory up a level (the parent directory).*

Example 4

Input	`path = "/../"`
Output	`"/"`
Explanation	*Going one level up from the root directory is not possible.*

QUESTION

Example 5

Input	`path = "/.../a/../b/c/../d/./"`
Output	`"/.../b/d"`
Explanation	*"..." is a valid name for a directory in this problem.*

- 1 <= path.length <= 3000
- path consists of English letters, digits, period '.', slash '/' or '_'.
- path is a valid absolute Unix path.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Simplify a Unix path: resolve '.' (current), '..' (parent), remove extra slashes. Right?"

"A few quick questions:

Multiple consecutive slashes -> single slash? Yes.

Result always starts with '/'? Yes. Never ends with '/' unless root."

"Let me walk: path='/home/../usr//bin/' -> '/usr/bin' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    replace '//' by '/', remove '/./', and remove
        'name/../' pairs
    if the string stops changing:
        break
drop a trailing slash
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each replacement rescans and copies the string, and there can be O(n) rounds. Split on '/' and use a stack (push names, pop on '..', ignore '.' and empty): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Stack-based path normalization — O(n). Walk-through:"

Split path by '/'. For each component:

'' or '.': skip (empty segment or current dir).

'..': pop from stack if non-empty (go up one level).

Else: push component (valid directory name).

Result = '/' + '/'.join(stack).

PHASE 3 OPTIMIZE

WHY stack: stack top = current directory level. '..' pops one level up.

WHY split by '/': multiple consecutive slashes produce empty strings -> filtered by skip.

Edge: root '/' -> stack is empty -> result is '/'. Handled naturally.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _71_SimplifyPath:
    def simplifyPath(self, path):
        stack = []
        for component in path.split('/'):
            if component == '..':
                if stack:
                    stack.pop()
            elif component and component != '.':
                stack.append(component)
        return '/' + '/'.join(stack)
```

PHASE 5 TEST & DEBUG

"Let me trace simplifyPath('/home/../foo//bar')."

Split by '/': ['','home','..','foo','','bar'].

Stack: process each part:

'': skip. 'home': push->['home']. '..': pop->[]. 'foo': push->['foo']. '': skip. 'bar': push->['foo','bar'].

Result: '/'+'/'.join(['foo','bar'])='/foo/bar' ok

'/home/': split->['','home',''']. process 'home'->['home']. skips. '/home' ok

'/...' at root: pop empty stack -> stay at root. Result: '/' ok

'/a//b': double slash -> empty string parts -> skipped ok

WHY stack: each '..' pops one level. '.' stays. Multiple slashes collapse to one.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
split + stack ops	stack stores path components

Follow-up: resolve relative paths given a current directory — prepend cwd to path.

Follow-up: Windows paths use backslash — replace '\' with '/' before processing.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #114 | 2 of 43 | 2/43

Flatten Binary Tree to Linked List

QUESTION

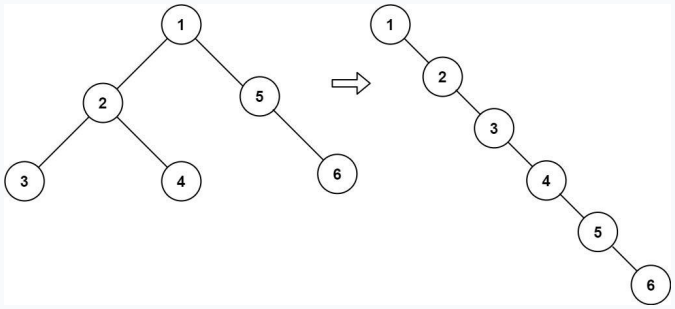
Given the root of a binary tree, flatten the tree into a "linked list":

CONSTRAINTS

- The "linked list" should use the same TreeNode class where the right child pointer points to the next node in the list and the left child pointer is always null.
- The "linked list" should be in the same order as a pre-order traversal of the binary tree.

Example 1

Input	`root = [1,2,5,3,4,null,6]`
Output	`[1,null,2,null,3,null,4,null,5,null,6]`



Example 2

Input	`root = []`
Output	`[]`

Example 3

Input	`root = [0]`
Output	`[0]`

- The number of nodes in the tree is in the range [0, 2000].
- 100 <= Node.val <= 100
- Follow up: Can you flatten the tree in-place (with O(1) extra space)?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Flatten a binary tree to a linked list in-place using preorder (root,left,right).

All right pointers, all left pointers = None. Right?"

"A few quick questions:

Modify in-place, no return value? Yes. Preorder order? Yes."

"Let me walk: root=[1,2,5,3,4,null,6] -> 1->2->3->4->5->6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
order = the pre-order traversal as a list of nodes
for each node set left = empty and right = the next
    node in order
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but stores all nodes. Rewiring in place (for each node with a left child, hook its right subtree under the rightmost node of the left subtree) is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is storing all nodes.

Morris traversal-like O(1) space approach:

For each node with a left subtree, find the rightmost node of its left subtree,

connect it to node.right, then move left subtree to right, set left=None.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _114_FlattenBinaryTree:
    def flatten(self, root):
        current = root
        while current:
            if current.left:
                rightmost = current.left
                while rightmost.right:
                    rightmost = rightmost.right
                rightmost.right = current.right
                current.right = current.left
                current.left = None
            current = current.right
```

PHASE 5 TEST & DEBUG

"Let me trace flatten on [1,2,5,3,4,null,6]."

Morris-like approach (right-threading):

At 1: has left child 2. Find rightmost of left subtree: 2->right=5, but 5 is right child of 1.

Wait — find rightmost of left(2): 2->right=None. prev=2... actually rightmost of left subtree = 4.

Let me retrace: 1 has left=2(left=3,right=4), right=5(right=6).

At 1: leftmost path is 2. Rightmost of left subtree (2): go right->4 (4 has no right).

PHASE 5 TEST & DEBUG

4.right = 1.right (=5). 1.right = 1.left (=2). 1.left = None.

Now: 1->2->3,right=4->5->6. Recurse/iterate on 1.right(=2):

At 2: left=3. Rightmost of left(3): 3 has no right, so 3.right=2.right(=4). 2.right=3. 2.left=None.

Continue: 1->2->3->4->5->6 ok

WHY rightmost of left then re-wire: classic Morris threading for in-place flattening.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each node's rightmost search is amortised O(1) per node total	**O(1)**

Follow-up: unflatten — reconstruct tree from the flattened list (needs preorder structure).

Follow-up: Increasing Order Search Tree (#897) — same idea but inorder, right-skewed.

MEDIUM | Stack | LeetCode #143 | 3 of 43 | 3/43

Reorder List

QUESTION

You are given the head of a singly linked-list. The list can be represented as:

$$L_0 \rightarrow L_1 \rightarrow \ldots \rightarrow L_{n-1} \rightarrow L_n$$

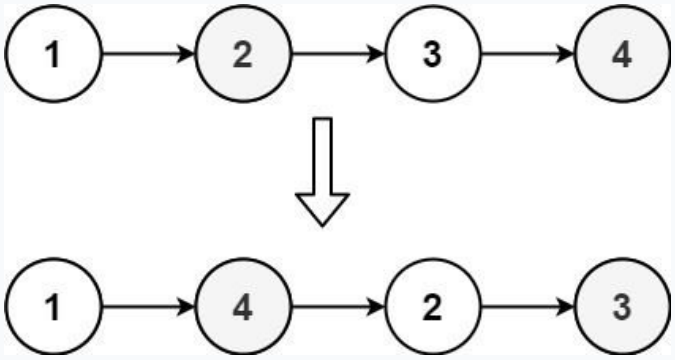
Reorder the list to be on the following form:

`L0 -> Ln -> L1 -> Ln - 1 -> L2 -> Ln - 2 -> …`

You may not modify the values in the list's nodes. Only nodes themselves may be changed.

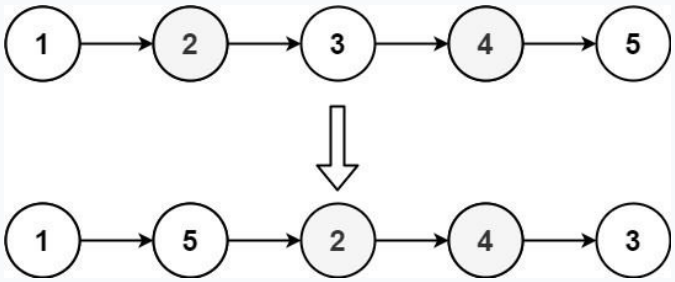
Example 1

Input	`head = [1,2,3,4]`
Output	`[1,4,2,3]`



Example 2

Input	`head = [1,2,3,4,5]`
Output	`[1,5,2,4,3]`



CONSTRAINTS

- The number of nodes in the list is in the range [1, 5 * 10^4].
- 1 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reorder in-place: L0->Ln->L1->Ln-1->L2->Ln-2->…

We swap the actual nodes, not just values. No return value. Right?"

PHASE 1 CLARIFY

"A few quick questions:

Can the list have 1 or 2 nodes? Yes — already in order, no-op.

O(1) space required? Yes — must not copy values or create new nodes."

"Let me walk: [1,2,3,4] -> [1,4,2,3] ok. [1,2,3,4,5] -> [1,5,2,4,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
nodes = all nodes in an array
i = 0, j = last
while i < j:
    link nodes[i] to nodes[j], then nodes[j] to
        nodes[i + 1]
    i = i + 1, j = j - 1
    cut the end
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but stores every node. Find the middle, reverse the second half in place, then merge the two halves: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary array of node references.

Three in-place steps: 1. Find the mid with slow/fast. 2. Reverse the second half.

3. Merge the two halves by interleaving.

Time: O(n). Space: O(1).

Does that make sense before I code it?"

Key insight: three in-place steps (find mid, reverse second half, interleave) each run in O(n) with O(1) space — no auxiliary array needed.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _143_ReorderList:
    def reorderList(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next; fast = fast.next.next

        prev, cur = None, slow
        while cur:
            nxt = cur.next; cur.next = prev; prev, cur = c
ur, nxt
        second = prev
```

```
        first = head
        while second.next:
            tmp1, tmp2 = first.next, second.next
            first.next = second; second.next = tmp1
            first, second = tmp1, tmp2
```

PHASE 5 TEST & DEBUG

"Let me trace reorderList(1->2->3->4->5)."

Step 1: find middle via slow/fast: mid=3. Second half=3->4->5.

Step 2: reverse second half: 5->4->3. First half: 1->2->3.

Step 3: merge alternately:

Iteration 1: l1=1, l2=5. l1.next=5, 5.next=2. l1=2, l2=4.

Iteration 2: l2=4. l1.next=4, 4.next=3. l1=3, l2=3.

Result: 1->5->2->4->3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
three linear passes	only pointer variables

This combines three classic linked-list techniques: mid-find, reverse, merge.

Follow-up: Palindrome Linked List (#234) uses the same first two steps.

Follow-up: if values-only (not node-pointer) swap allowed, the brute approach is simpler.

MEDIUM | Stack | LeetCode #150 | 4 of 43 | 4/43

Evaluate Reverse Polish Notation

QUESTION

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

CONSTRAINTS

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1

Input	`tokens = ["2","1","+","3","*"]`
Output	**9**
Explanation	*((2 + 1) * 3) = 9*

Example 2

Input	`tokens = ["4","13","5","/","+"]`
Output	**6**
Explanation	*(4 + (13 / 5)) = 6*

Example 3

Input	`tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]`
Output	**22**
Explanation	*((10 * (6 / ((9 + 3) * -11))) + 17) + 5* *= ((10 * (6 / (12 * -11))) + 17) + 5* *= ((10 * (6 / -132)) + 17) + 5* *= ((10 * 0) + 17) + 5* *= (0 + 17) + 5* *= 17 + 5* *= 22*

- $1 <= tokens.length <= 10^4$
- tokens[i] is either an operator: "+", "-", "*", or "/", or an integer in the range [-200, 200].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Evaluate an expression in Reverse Polish Notation: operands push onto a stack,

operators pop two, compute, and push result. Division truncates toward zero. Right?"

"A few quick questions:

Only +, -, *, /? Yes. Can intermediate results overflow 32-bit? Python handles big ints natively.

Division truncates toward zero — Python's // floors toward negative infinity, so I need int(a/b)."

"Let me walk: ['2','1','+','3','*'] -> (2+1)*3 = 9 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
tokens = a list
while True:
    find the first operator, apply it to the two
        numbers just before it, replace those
        three tokens by the result
    if one token is left:
        break
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Finding the operator and replacing three list entries costs O(n) per step. One left-to-right pass with a stack (push numbers, pop two on an operator) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the repeated linear scan.

A stack processes this in a single left-to-right pass:

push operands; on each operator, pop two, compute, push result.

Final stack element is the answer.

Time: O(n). Space: O(n) for the stack.

Does that make sense before I code it?"

Key insight: the stack models a postfix calculator's operand register — each token either pushes a value or pops two, computes, and pushes the result; every token is processed exactly once.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _150_EvalRPN:
    def evalRPN(self, tokens):
        stack = []
        ops = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b),
        }
        for token in tokens:
            if token in ops:
                b, a = stack.pop(), stack.pop()
                stack.append(ops[token](a, b))
            else:
                stack.append(int(token))
        return stack[0]
```

PHASE 5 TEST & DEBUG

"Let me trace evalRPN(['2','1','+','3',''])."*

stack=[]. '2'->push->[2]. '1'->push->[2,1]. '+'->pop 1 and 2->push 3->[3].

'3'->push->[3,3]. '*'->pop 3 and 3->push 9->[9].

Return 9 ok ((2+1)*3=9).

['4','13','5','/','+']:

push 4. push 13. push 5. '/'->pop 5,13->13//5=2->push 2. stack=[4,2]. '+'->pop 2,4->6.

Return 6 ok (4+(13/5)).

WHY stack: RPN evaluates left-to-right. When operator seen, pop last two operands.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
single pass	at most (n+1)/2 operands on stack simultaneously

The lambda dict keeps the operator dispatch clean and avoids four if-elif branches.

Follow-up: Basic Calculator (#224) — infix with parentheses; needs operator precedence stack.

Follow-up: validate an RPN expression — operand count must always exceed operator count.

MEDIUM | Stack | LeetCode #155 | 5 of 43 | 5/43

Min Stack

QUESTION

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

CONSTRAINTS

- MinStack() initializes the stack object.
- void push(int val) pushes the element val onto the stack.
- void pop() removes the element on the top of the stack.
- int top() gets the top element of the stack.
- int getMin() retrieves the minimum element in the stack.

You must implement a solution with O(1) time complexity for each function.

- 2^{31} <= val <= 2^{31} - 1
- Methods pop, top and getMin operations will always be called on non-empty stacks.
- At most 3 * 10^4 calls will be made to push, pop, top, and getMin.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a stack that supports push, pop, top, and getMin — all O(1). Right?"

"A few quick questions:

getMin must return the current minimum, which can change as we push/pop? Yes.

pop and top are only called on non-empty stacks? Yes."

"Let me walk: push(-2),push(0),push(-3). getMin->-3. pop(). top()->0. getMin()->-2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result.append(a_normal_stack)
def getMin():
    scan every element and return the smallest
```

TIME	SPACE
O(n)	O(1)
for getMin	extra

WHY IT IS SLOW

getMin walks the whole stack. Storing (value, minimum so far) with each push makes push, pop, top and getMin all O(1).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) scan for getMin.

Maintain a parallel 'min stack' tracking the minimum at each level:

when we push val, also push min(val, min_stack.top()) to the min stack.

When we pop, pop both stacks together.

getMin always reads min_stack.top() — O(1).

Time: O(1) all ops. Space: O(n) for the two stacks.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _155_MinStack:
    def __init__(self):
        self._stack = []
        self._min_stack = []

    def push(self, val):
        self._stack.append(val)
        current_min = val if not self._min_stack else min(
val, self._min_stack[-1])
        self._min_stack.append(current_min)

    def pop(self):
        self._stack.pop()
        self._min_stack.pop()

    def top(self): return self._stack[-1]
    def getMin(self): return self._min_stack[-1]
```

PHASE 5 TEST & DEBUG

"Let me trace MinStack: push(-2),push(0),push(-3),getMin(),pop(),getMin()."

Each node stores (val, current_min). Stack of pairs.

```
push(-2): min=min(inf,-2)=-2. stack=[(-2,-2)].
push(0): min=min(-2,0)=-2. stack=[(-2,-2),(0,-2)].
push(-3): min=min(-2,-3)=-3.
  stack=[(-2,-2),(0,-2),(-3,-3)].
getMin(): stack.top().min=-3 ok
pop(): remove (-3,-3). stack=[(-2,-2),(0,-2)].
getMin(): stack.top().min=-2 ok
```

WHY store min per frame: after pop, instantly recover prior min — no O(n) rescan.

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(1). Space O(n): two stacks of size n.

Alternative: store (val, current_min) as a tuple in one stack — halves number of lists.

Follow-up: Max Stack (#716) — same parallel-stack idea with a max_stack.

Follow-up: if push is frequent but getMin rare, brute O(n) getMin is acceptable.

MEDIUM | Stack | LeetCode #173 | 6 of 43 | 6/43

Binary Search Tree Iterator

QUESTION

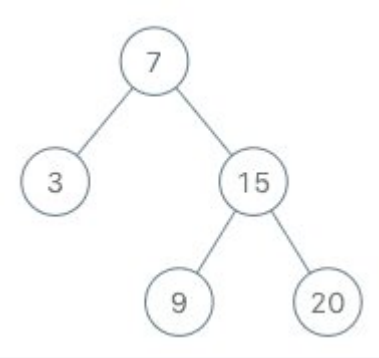
Implement the BSTIterator class that represents an iterator over the in-order traversal of a binary search tree (BST):

CONSTRAINTS

- BSTIterator(TreeNode root) Initializes an object of the BSTIterator class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- boolean hasNext() Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- int next() Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to next() will return the smallest element in the BST.

You may assume that next() calls will always be valid. That is, there will be at least a next number in the in-order traversal when next() is called.



- The number of nodes in the tree is in the range [1, 10^5].
- 0 <= Node.val <= 10^6
- At most 10^5 calls will be made to hasNext, and next.
- Follow up:
- Could you implement next() and hasNext() to run in average O(1) time and use O(h) memory, where h is the height of the tree?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement an iterator over a BST. next() returns the next smallest element,

`hasNext() returns true if more elements remain.`

Both must average O(1) time and use O(h) space where h is the tree height. Right?"

"A few quick questions:

`next() is always called when an element exists? Yes — per constraints.`

O(h) space means we can use a stack of depth h?"

PHASE 1 CLARIFY

"Let me walk: BST=[7,3,15,null,null,9,20]. next()->3,7,9,15,20 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
at construction:
    values = the full in-order traversal in a list
def next():
    return values[pointer++]
def hasNext():
    pointer < len
```

TIME	SPACE
O(n) to build, O(1) per call	**O(n)**

WHY IT IS SLOW

Fast calls but O(n) memory up front. A stack holding only the left spine (pop, then push the left spine of the right child) gives O(1) amortised next and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) pre-stored list.

Controlled inorder traversal using an explicit stack — O(h) space:

At init, push the entire left spine.

`next(): pop the top node (smallest available), push its right subtree's left spine.`

Each node is pushed and popped exactly once — amortized O(1) per next() call.

Time: O(1) amortized. Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _173_BSTIterator:
    def __init__(self, root):
        self._stack = []
        self._push_left(root)

    def _push_left(self, node):
        while node:
            self._stack.append(node)
            node = node.left

    def next(self):
        node = self._stack.pop()
        self._push_left(node.right)
        return node.val

    def hasNext(self):
        return bool(self._stack)
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

BST=[7,3,15,null,null,9,20]: init pushes [7,3] (left spine). stack=[7,3].

next(): pop 3, push 3.right=None -> nothing. return 3. stack=[7].

next(): pop 7, push left spine of 15 -> [15,9]. return 7. stack=[15,9].

next(): pop 9, push 9.right=None. return 9. stack=[15].

hasNext()->True ok. next(): pop 15, push [20]. return 15. next(): pop 20. return 20 ok

_push_left handles None safely with the 'while node' guard

PHASE 6 COMPLEXITY & FOLLOW-UP

next(): amortized O(1) — each node pushed/popped once across all calls.

hasNext(): O(1). Space: O(h) — left spine depth.

Follow-up: reverse inorder iterator (descending) — push right spines instead.

Follow-up: peek() without advancing — call next() then push the value back as a synthetic node.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #227 | 7 of 43 | 7/43

Basic Calculator II

QUESTION

Given a string s which represents an expression, evaluate this expression and return its value.

The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Example 1	
Input	`s = "3+2*2"`
Output	7

Example 2	
Input	`s = " 3/2 "`
Output	1

Example 3	
Input	`s = " 3+5 / 2 "`
Output	5

CONSTRAINTS

- $1 <= s.length <= 3 * 10^5$
- s consists of integers and operators ('+', '-', '*', '/') separated by some number of spaces.
- s represents a valid expression.
- All the integers in the expression are non-negative integers in the range $[0, 2^{31} - 1]$.
- The answer is guaranteed to fit in a 32-bit integer.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Evaluate a string with +, -, *, / (no parentheses), non-negative integers, possible spaces.

Division truncates toward zero. No built-in eval. Right?"

"A few quick questions:

Can the expression start with a negative number? No — all integers are non-negative.

Spaces can appear anywhere between tokens?"

"Let me walk: '3+2*2' -> 2*2=4 first, then 3+4=7 ok. ' 3/2 '->1 ok. '3+5/2'->5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
first resolve every '*' and '/' by finding the
    operator, computing its two neighbours and
    replacing them in the string
then resolve '+' and '-' left to right
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each replacement rebuilds the string. One pass with a stack: push numbers (negate for '-'), and combine immediately for '*' and '/'. Then sum the stack: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the two-pass tokenisation.

Single-pass stack: track the last operator (sign).

On + or -: push the current number with its sign.

On * or /: pop the top, compute immediately with the current number, push result.

Final answer: sum the stack.

Time: O(n). Space: O(n) for the stack."

Key insight: computing +/- operands immediately and stacking only multiplication/division operands avoids two-pass evaluation — one left-to-right scan with a signed running total.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _227_BasicCalculatorII:
    def calculate(self, s):
        stack, num, sign = [], 0, '+'
        for i, ch in enumerate(s):
            if ch.isdigit():
                num = num * 10 + int(ch)
            if (ch in '+-*/') or i == len(s) - 1:
                if sign == '+': stack.append(num)
                elif sign == '-': stack.append(-num)
                elif sign == '*': stack.append(stack.pop()
 * num)
                else: stack.append(int(stack.pop() / num))
                sign, num = ch, 0
        return sum(stack)
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

'3+2*2':

'3': num=3. '+': sign was '+' -> push 3->[3]. sign='+',num=0.

'2': num=2. '*': sign='+' -> push 2->[3,2]. sign='*',num=0.

'2'(last): num=2. i=end: sign='*' -> pop 2*2=4 -> [3,4]. sum=7 ok

' 3/2 ': num=3. '/': push 3->[3]. num=2 (last): int(3/2)=1->[1]. sum=1 ok

Spaces are skipped because only digits and operators trigger action

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
one pass	stack holds at most n/2 numbers

Key: * and / are computed immediately (higher precedence); + and - defer to the final sum.

Follow-up: Basic Calculator I (#224) — adds parentheses; handle with a recursion/stack.

Follow-up: Basic Calculator III (#772) — all four ops plus parentheses.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #255 | 8 of 43 | 8/43

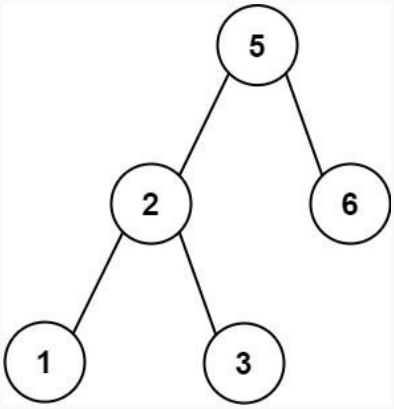
Verify Preorder Sequence in Binary Search Tree

QUESTION

Given an array of unique integers preorder, return true if it is the correct preorder traversal sequence of a binary search tree.

Example 1

Input	`preorder = [5,2,1,3,6]`
Output	`true`



Example 2

Input	`preorder = [5,2,6,1,3]`
Output	`false`

CONSTRAINTS

- 1 <= preorder.length <= 10^4
- 1 <= preorder[i] <= 10^4
- All the elements of preorder are unique.
- Follow up: Could you do it using only constant space complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an array of unique integers, determine if it could be a valid preorder traversal sequence of a binary search tree. Right?"

"A few quick questions:

Preorder: root, then left subtree (all smaller), then right subtree (all larger).

Once we enter the right subtree, all subsequent nodes must be larger than the root?"

"Let me walk: [5,2,1,3,6]: root=5, left=[2,1,3], right=[6] — valid ok

[5,2,6,1,3]: after right subtree starts (6>5), seeing 1<5 is invalid -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def check(lo, hi):
    root = pre[lo]
    split = the first index after lo with a value
        greater than root
    if any value at or after split is below root:
        return False
    return check(lo + 1, split - 1) and
        check(split, hi)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Rescans each segment at every level, O(n^2) for a sorted input. A single pass with a stack and a lower bound (every value must exceed the last popped ancestor) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n²) recursion on skewed trees.

Monotonic stack + lower bound tracking — O(n) time, O(n) space:

Maintain a stack of nodes on the left-spine (descending values).

When we see a value greater than stack top, we've entered a right subtree —

pop all smaller values, the last popped becomes the new lower bound.

Any subsequent value must exceed this lower bound.

Time: O(n). Space: O(n) for the stack."

Key insight: the lower_bound variable tracks the minimum allowed value in the current right subtree — any node violating it proves the sequence isn't valid BST preorder.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _255_VerifyPreorderBST:
    def verifyPreorder(self, preorder):
        stack = []
        lower_bound = float('-inf')
        for val in preorder:
            if val < lower_bound:
                return False
            while stack and stack[-1] < val:
                lower_bound = stack.pop()
            stack.append(val)
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

[5,2,1,3,6]:

5: stack=[5]. 2<5: stack=[5,2]. 1<2: stack=[5,2,1].

3>1: pop1(lb=1), pop2(lb=2). 3<5 stop. stack=[5,3]. 3>lb=2 ok.

6>5: pop3(lb=3), pop5(lb=5). stack=[6]. 6>lb=5 ok. -> True ok

[5,2,6,1,3]:

5->[5]. 2->[5,2]. 6>2: pop2(lb=2),pop5(lb=5). stack=[6]. 6>lb=5 ok.

1<lb=5 -> False ok

The lower_bound tracks the minimum acceptable value after entering a right subtree

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element pushed and popped at most once	stack

O(1) space variant: reuse the preorder array itself as the stack (destructive).

Follow-up: reconstruct BST from preorder (#1008) — similar range-based recursion.

Follow-up: verify inorder — just check it's strictly sorted.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #316 | 9 of 43 | 9/43

Remove Duplicate Letters

QUESTION

Given a string s, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Example 1	
Input	`s = "bcabc"`
Output	`"abc"`

Example 2	
Input	`s = "cbacdcbc"`
Output	`"acdb"`

CONSTRAINTS

- 1 <= s.length <= 10^4
- s consists of lowercase English letters.
- Note: This question is the same as 1081: https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove duplicate letters so each letter appears once, in the smallest lexicographic order,

maintaining the relative order of remaining letters. Right?"

"A few quick questions:

Must use all unique letters (can't omit any)? Yes — all distinct chars must appear once."

"Let me walk: s='bcabc' -> 'abc' ok. s='cbacdcbc' -> 'acdb' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subsequence in subsequences:
    # of s with each letter exactly once
    best = min(best, candidate)
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Enumerates exponentially many subsequences. A monotonic stack (pop a larger letter if it appears again later and the current one is not already in the stack) with last-occurrence indexes is O(n).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the exponential search.

Greedy monotonic stack: for each character, pop the stack top if:

1. Current char is smaller (lex), AND

2. The top char still appears later (count[top] > 0).

If current char is already in stack, skip it.

Time: O(n). Space: O(26) = O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import Counter
class _316_RemoveDuplicateLetters:
    def removeDuplicateLetters(self, s):
        remaining = Counter(s)
        in_stack = set()
        stack = []
        for ch in s:
            remaining[ch] -= 1
            if ch in in_stack:
                continue
            while stack and ch < stack[-1] and remaining[s
tack[-1]] > 0:
                in_stack.discard(stack.pop())
            stack.append(ch)
            in_stack.add(ch)
        return ''.join(stack)
```

PHASE 5 TEST & DEBUG

"Let me trace removeDuplicateLetters('bcabc')."

count={b:2,c:2,a:1}. in_stack={}.stack=[].

'b': stack=['b']. count[b]=1. in_stack={b}.

'c': stack=['b','c']. count[c]=1. in_stack={b,c}.

'a': a<c AND count[c]=1>0 -> DON'T pop c (need it later, count still>0? No: count[c]=1 means 1 more ahead).

Wait count tracks remaining occurrences ahead. 'a': stack top='c'. a<c AND count[c]=1>0 -> pop c (in_stack remove c). stack=['b']. b>a? No a<b AND count[b]=1>0 -> pop b. stack=[]. push a. stack=['a'].

'b': a<b -> push. stack=['a','b']. count[b]=0.

'c': b<c -> push. stack=['a','b','c']. count[c]=0.

Return 'abc' ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each char pushed/popped at most once	O(26)

PHASE 6 COMPLEXITY & FOLLOW-UP

This is also called 'Smallest Subsequence of Distinct Characters' (#1081) — identical problem.

Follow-up: if we could repeat characters (not require distinct), greedy still applies.

Follow-up: Largest Rectangle in Histogram (#84) — similar monotonic stack pattern.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #394 | 10 of 43 | 10/43

Decode String

QUESTION

Given an encoded string, return its decoded string.

The encoding rule is: k[encoded_string], where the encoded_string inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k. For example, there will not be input like 3a or 2[4].

The test cases are generated so that the length of the output will never exceed 10^5.

Example 1	
Input	`s = "3[a]2[bc]"`
Output	`"aaabcbc"`

Example 2	
Input	`s = "3[a2[c]]"`
Output	`"accaccacc"`

Example 3	
Input	`s = "2[abc]3[cd]ef"`
Output	`"abcabccdcdcdef"`

CONSTRAINTS

- 1 <= s.length <= 30
- s consists of lowercase English letters, digits, and square brackets '[]'.
- s is guaranteed to be a valid input.
- All the integers in s are in the range [1, 300].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Decode 'k[encoded_string]' — repeat encoded_string k times.

Nesting is allowed. k is always a positive integer. Input is always valid. Right?"

"A few quick questions:

Can k be multi-digit? Yes — e.g. '12[a]' -> 'aaa...a'(12 times).

Can encoded_string be empty? Yes — e.g. '3[]' -> ""

"Let me walk: '3[a]2[bc]' -> 'aaabcbc' ok. '3[a2[c]]' -> 'accaccacc' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while the string contains ']':
    find the first ']', its matching '[' and the
        number before it
    replace k[x] by x repeated k times
```

TIME	SPACE
O(L * d)	O(L)

WHY IT IS SLOW

Rebuilds the whole string once per bracket pair (L = output length, d = nesting). One recursive pass, or a stack of (text so far, repeat count), builds the answer in O(L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the repeated string scans.

Stack-based single pass:

On digit: build the multiplier k (can be multi-digit).

On '[': push (current_string, k) onto the stack, reset both to empty/0.

On ']': pop (prev_string, k), set current = prev_string + k * current.

On letter: append to current string.

Time: O(n * max_k) for building the expanded output. Space: O(n) stack depth."

Key insight: on '[' push (current_result, multiplier) to freeze the outer context; on ']' pop and prepend the outer result — nested encodings unwind naturally in reverse order.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _394_DecodeString:
    def decodeString(self, s):
        stack = []
        current = ''
        k = 0
        for ch in s:
            if ch.isdigit():
                k = k * 10 + int(ch)
            elif ch == '[':
                stack.append((current, k))
                current, k = '', 0
            elif ch == ']':
                prev, repeat = stack.pop()
                current = prev + repeat * current
            else:
                current += ch
        return current
```

PHASE 5 TEST & DEBUG

"Let me trace decodeString('3[a2[c]]')."

stack=[]. result=''. k=0.

'3': k=3. '[': push(result='',k=3). result='',k=0.

'a': result='a'. '2': k=2. '[': push(result='a',k=2). result='',k=0.

'c': result='c'. ']': pop->(prev='a',times=2). result='a'+'c'*2='acc'.

']': pop->(prev='',times=3). result=''+'acc'*3='accaccacc'.

Return 'accaccacc' ok

'2[abc]3[cd]ef': [2,'abc'] -> 'abcabc'. [3,'cd'] -> 'cdcdcd'. Append 'ef'. Return 'abcabccdcdcdef' ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * max_k)	**O(d)**
building the output string can be large	d = nesting depth of the stack

Follow-up: Encode String with Shortest Length (#471) — reverse direction, DP + KMP.

Follow-up: if nesting is guaranteed absent, a simple split-on-bracket regex suffices.

MEDIUM | Stack | LeetCode #402 | 11 of 43 | 11/43

Remove K Digits

QUESTION

Given string num representing a non-negative integer num, and an integer k, return the smallest possible integer after removing k digits from num.

Example 1	
Input	`num = "1432219", k = 3`
Output	`"1219"`
Explanation	*Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.*

Example 2	
Input	`num = "10200", k = 1`
Output	`"200"`
Explanation	*Remove the leading 1 and the number is 200. Note that the output must not contain leading zeroes.*

Example 3	
Input	`num = "10", k = 2`
Output	`"0"`
Explanation	*Remove all the digits from the number and it is left with nothing which is 0.*

CONSTRAINTS

- $1 <= k <= num.length <= 10^5$
- num consists of only digits.
- num does not have any leading zeros except for the zero itself.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove k digits from a number string to make the smallest possible number.

No leading zeros in result (except '0' itself). Right?"

"A few quick questions:

Result can be empty -> return '0'? Yes."

"Let me walk: num='1432219', k=3 -> remove 4,3,2 -> '1219' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for way in ways:
    # to choose k digits to remove
    form the number
    best = min(best, candidate)
```

TIME	SPACE
O(C(n, k) * n)	O(n)

WHY IT IS SLOW

Enumerates all C(n, k) removals. A monotonic stack that pops a larger digit whenever a smaller one arrives (while removals remain) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack greedy — O(n). Walk-through:"

stack=[]. removals=k.

For each digit d in num:

While removals>0 and stack and stack[-1]>d:

`stack.pop(); removals--. <- remove larger digit that preceded smaller digit`

`stack.append(d).`

Remove from end if removals>0: stack = stack[:-removals].

Strip leading zeros. Return ''.join(stack) or '0'.

WHY remove larger digit before smaller: keeping a smaller digit earlier gives a smaller number.

'1432' with k=1: remove 4 (first digit > next) -> '132'. Greedy: remove leftmost peak.

WHY remove from end if k>0: all remaining digits are non-decreasing -> remove from end.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _402_RemoveKDigits:
    def removeKdigits(self, num, k):
        stack = []
        for digit in num:
            while k > 0 and stack and stack[-1] > digit:
                stack.pop()
                k -= 1
            stack.append(digit)
        final = stack[:len(stack) - k] if k else stack
        result = ''.join(final).lstrip('0')
        return result or '0'
```

PHASE 5 TEST & DEBUG

"Let me trace removeKdigits('1432219', k=3)."

Monotonic stack (keep smallest possible leading digits).

stack=[]. k=3.

'1': push -> [1]. '4': push -> [1,4]. '3': 4>3 AND k>0 -> pop 4, k=2. Push 3 -> [1,3].

'2': 3>2 AND k>0 -> pop 3, k=1. 2>2? No. Push 2 -> [1,2].

'2': 2==2 -> push -> [1,2,2].

'1': 2>1 AND k>0 -> pop 2, k=0. stack=[1,2,1]. Push 1... Wait k=0 now so no more pops.

Actually: after popping 3,2,1 (k decrements each pop):

Result trace: '1': [1]. '4': [1,4]. '3': pop4(k=2),push3 -> [1,3]. '2': pop3(k=1),push2 -> [1,2]. '2': [1,2,2]. '1': pop2(k=0),NO more pops,push1 -> [1,2,1]. '9': [1,2,1,9].

Join, strip leading zeros. k=0 so no trimming needed. Return '1219' ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each digit pushed/popped once	stack

Follow-up: Largest Number After Digit Swaps by Parity (#2231) — similar greedy.

Follow-up: Create Maximum Number (#321) — combine two arrays, same monotonic stack idea.

MEDIUM | Stack | LeetCode #426 | 12 of 43 | 12/43

Convert Binary Search Tree to Sorted Doubly Linked List

QUESTION

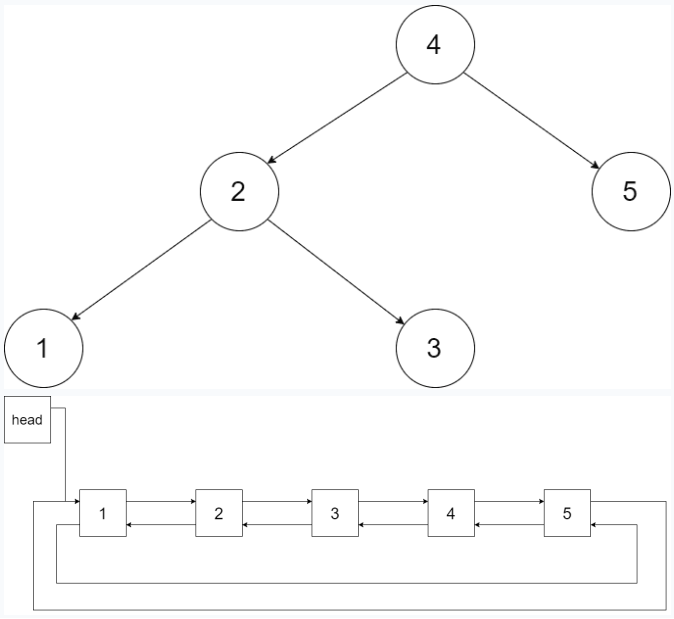
Convert a Binary Search Tree to a sorted Circular Doubly-Linked List in place.

You can think of the left and right pointers as synonymous to the predecessor and successor pointers in a doubly-linked list. For a circular doubly linked list, the predecessor of the first element is the last element, and the successor of the last element is the first element.

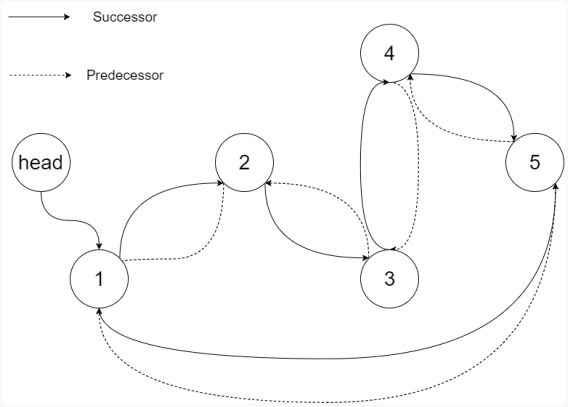
We want to do the transformation in place. After the transformation, the left pointer of the tree node should point to its predecessor, and the right pointer should point to its successor. You should return the pointer to the smallest element of the linked list.

Example 1

Input	`root = [4,2,5,1,3]`
Output	`[1,2,3,4,5]`
Explanation	*The figure below shows the transformed BST. The solid line indicates the successor relationship, while the dashed line means the predecessor relationship.*



QUESTION



Example 2

Input	`root = [2,1,3]`
Output	`[1,2,3]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 2000].
- 1000 <= Node.val <= 1000
- All the values of the tree are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Convert a BST to a sorted circular doubly linked list in-place.

The left pointer acts as prev, right as next. Right?"

"A few quick questions:

In-place modification — no new nodes? Yes. Smallest and largest nodes are linked circularly? Yes."

"Let me walk: BST [4,2,5,1,3] -> DLL: 12345(back to 1) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the in-order traversal as a list of nodes
link each node to its neighbours in the list and
    close the circle
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but stores every node. An in-order traversal that keeps only the previous node and relinks as it goes uses O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) list storage.

Inorder traversal with in-place pointer wiring:

Track prev pointer; link prev.right=curr and curr.left=prev on each visit.

After full traversal, link head and tail for circularity.

Time: O(n). Space: O(h)."

Key insight: inorder traversal visits BST nodes in sorted order — threading prev.right = curr and curr.left = prev in-place during traversal builds the doubly-linked list with O(1) extra space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _426_ConvertBSTToSortedDoublyLinkedList:
    def treeToDoublyList(self, root):
        if not root: return None
        self_head = [None]
        self_prev = [None]
        def inorder(node):
            if not node: return
            inorder(node.left)
            if self_prev[0]:
                self_prev[0].right = node
                node.left = self_prev[0]
            else:
                self_head[0] = node
            self_prev[0] = node
            inorder(node.right)
        inorder(root)
        self_head[0].left = self_prev[0]
        self_prev[0].right = self_head[0]
        return self_head[0]
```

PHASE 5 TEST & DEBUG

"Let me trace treeToDoublyList on BST [4,2,5,1,3]."

In-order (L-N-R) = 1,2,3,4,5.

head=None, prev=None.

Visit 1: prev=None->head=1. prev=1.

Visit 2: prev.right=2, 2.left=prev=1. prev=2.

Visit 3: prev.right=3, 3.left=prev=2. prev=3.

Visit 4: prev.right=4, 4.left=prev=3. prev=4.

Visit 5: prev.right=5, 5.left=prev=4. prev=5.

Connect circular: head.left=prev=5. prev.right=head=1. -> 12345(circular) ok

WHY in-order: BST in-order = sorted ascending = correct DLL order.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** for recursion

Follow-up: convert DLL back to BST — find mid, split, recurse.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: merge two sorted DLLs — merge like merge sort then convert to BST.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #439 | 13 of 43 | 13/43

Ternary Expression Parser

QUESTION

Given a string expression representing arbitrarily nested ternary expressions, evaluate the expression, and return the result of it.

You can always assume that the given expression is valid and only contains digits, '?', ':', 'T', and 'F' where 'T' is true and 'F' is false. All the numbers in the expression are one-digit numbers (i.e., in the range [0, 9]).

The conditional expressions group right-to-left (as usual in most languages), and the result of the expression will always evaluate to either a digit, 'T' or 'F'.

Example 1

Input	`expression = "T?2:3"`
Output	`"2"`
Explanation	*If true, then result is 2; otherwise result is 3.*

Example 2

Input	`expression = "F?1:T?4:5"`
Output	`"4"`
Explanation	*The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:* *"(F ? 1 : (T ? 4 : 5))" --> "(F ? 1 : 4)" --> "4"* *or "(F ? 1 : (T ? 4 : 5))" --> "(T ? 4 : 5)" --> "4"*

Example 3

Input	`expression = "T?T?F:5:3"`
Output	`"F"`
Explanation	*The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:* *"(T ? (T ? F : 5) : 3)" --> "(T ? F : 3)" --> "F"* *"(T ? (T ? F : 5) : 3)" --> "(T ? F : 5)" --> "F"*

CONSTRAINTS

- 5 <= expression.length <= 10^4
- expression consists of digits, 'T', 'F', '?', and ':'.
- It is guaranteed that expression is a valid ternary expression and that each number is a one-digit number.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Evaluate a ternary expression string: 'T?a:b' -> a if T, else b.

Can be arbitrarily nested. All conditions are T or F; values are digits, T, or F.

Ternary is right-associative. Right?"

PHASE 1 CLARIFY

"A few quick questions:

Input is always a valid ternary? Yes. Only single-char values? Yes (digits 0-9, T, F)."

"Let me walk: 'T?2:3' -> '2' ok. 'F?1:T?4:5' -> '4' ok. 'T?T?F:5:3' -> 'F' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def eval(expr):
    if it is a single digit or letter:
        return it
    find the first '?' and its matching ':' by
        counting
    return eval(true part) if the condition is
        'T' else eval(false part)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Rescans the string to find the matching ':' at each nesting level. Scan right to left with a stack: when you meet '?', the top two entries are the branches, pick one by the condition: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the repeated string scans.

Scan RIGHT TO LEFT with a stack — O(n):

When we see '?': pop two values (true-branch then false-branch) and the condition character.

Push the resolved value.

Right-to-left processes the right-associativity naturally.

Time: O(n). Space: O(n) stack."

Key insight: scanning right-to-left with a stack processes nested ternaries outside-in — each '?' pops two branches and pushes the selected value; the stack naturally unwinds nesting in O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _439_TernaryExpressionParser:
    def parseTernary(self, expression):
        stack = []
        for ch in reversed(expression):
            if stack and stack[-1] == '?':
                stack.pop()
                true_val = stack.pop()
                stack.pop()
                false_val = stack.pop()
                stack.append(true_val if ch == 'T' else fa
```

```
lse_val)
            else:
                stack.append(ch)
        return stack[0]
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

'T?2:3' reversed='3:2?T':

push '3'. ':' -> push ':'. '2' -> push '2'. '?' -> pop '?', true='2', pop ':', false='3'.

'T' -> push '2'. stack=['2']. return '2' ok

'T?T?F:5:3' (right-assoc is T?(T?F:5):3):

Process right-to-left: resolves inner T?F:5 first -> F, then outer T?F:3 -> F ok

Popping ':' after the false_val keeps the separator from polluting the stack

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
one right-to-left pass	stack depth

The right-to-left scan makes right-associativity automatic — no parenthesis tracking needed.

Follow-up: if operands can be multi-character, tokenize first then apply the same logic.

Follow-up: support general conditional expressions (==, !=) — extend the condition parsing.

MEDIUM | Stack | LeetCode #445 | 14 of 43 | 14/43

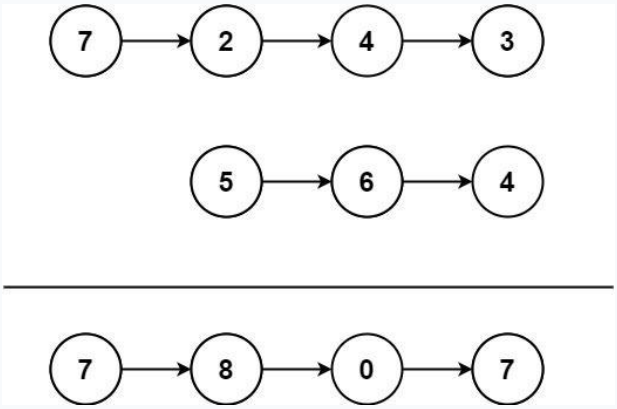
Add Two Numbers II

QUESTION

You are given two non-empty linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1	
Input	`l1 = [7,2,4,3], l2 = [5,6,4]`
Output	`[7,8,0,7]`



Example 2	
Input	`l1 = [2,4,3], l2 = [5,6,4]`
Output	`[8,0,7]`

Example 3	
Input	`l1 = [0], l2 = [0]`
Output	`[0]`

CONSTRAINTS

- The number of nodes in each linked list is in the range [1, 100].
- 0 <= Node.val <= 9
- It is guaranteed that the list represents a number that does not have leading zeros.
- Follow up: Could you solve it without reversing the input lists?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two numbers stored in linked lists with MOST significant digit first. Return their sum as a list. Right?"

"A few quick questions:

No leading zeros except the number 0 itself? Yes.

This is the REVERSE of #2 — digits are forward order."

PHASE 1 CLARIFY

"Let me walk: l1=[7,2,4,3], l2=[5,6,4] -> 7243+564=7807 -> [7,8,0,7] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
turn each list into an integer
add them
turn the result back into a list
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

The numbers can have 100 digits, which overflows fixed-width integers. Push both lists onto stacks and add from the least significant end with a carry, building the result by prepending nodes.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Reverse lists + grade-school addition + reverse result — O(n). Walk-through:"

Reverse both linked lists (so LSB (Least Significant Bit) is first).

Add digit-by-digit with carry: while l1 or l2 or carry:

`s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry.`

`carry = s // 10. new digit = s % 10.`

Append to result list.

Advance l1, l2.

Reverse result list (so MSB (Most Significant Bit) is first again).

WHY reverse: addition must start from LSB (rightmost). Reversing allows iterating from LSB.

WHY carry after loop ends: if both lists exhausted and carry=1, need one more digit.

Alternative: use stacks to process MSB lists from LSB without reversing pointers.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _445_AddTwoNumbersII:
    def addTwoNumbers(self, l1, l2):
        stack1, stack2 = [], []
        cur = l1
        while cur: stack1.append(cur.val); cur = cur.next
        cur = l2
        while cur: stack2.append(cur.val); cur = cur.next
        carry = 0
```

```
        head = None
        while stack1 or stack2 or carry:
            d1 = stack1.pop() if stack1 else 0
            d2 = stack2.pop() if stack2 else 0
            total = d1 + d2 + carry
            carry, digit = divmod(total, 10)
            node = ListNode(digit)
            node.next = head
            head = node
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

l1=[7,2,4,3], l2=[5,6,4]: stacks=[7,2,4,3],[5,6,4].

pop 3+4=7 carry=0->node(7). pop 4+6=10->carry=1,node(0). pop 2+5+1=8->node(8). pop 7->node(7).

head->7->8->0->7 ok

Prepending nodes builds the result in correct (most-significant-first) order

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m)	**O(n+m)** two stacks

Reverse both lists + apply #2 algorithm is equivalent — same complexity.

Follow-up: Add Two Numbers I (#2) — least significant first; no stacks needed.

Follow-up: if lists have very different lengths, padding with zeros is implicit via 'else 0'.

MEDIUM | Stack | LeetCode #456 | 15 of 43 | 15/43

132 Pattern

QUESTION

Given an array of n integers nums, a 132 pattern is a subsequence of three integers nums[i], nums[j] and nums[k] such that i < j < k and nums[i] < nums[k] < nums[j].

Return true if there is a 132 pattern in nums, otherwise, return false.

Example 1	
Input	`nums = [1,2,3,4]`
Output	`false`
Explanation	*There is no 132 pattern in the sequence.*

Example 2	
Input	`nums = [3,1,4,2]`
Output	`true`
Explanation	*There is a 132 pattern in the sequence: [1, 4, 2].*

Example 3	
Input	`nums = [-1,3,2,0]`
Output	`true`
Explanation	*There are three 132 patterns in the sequence: [-1, 3, 2], [-1, 3, 0] and [-1, 2, 0].*

CONSTRAINTS

- n == nums.length
- 1 <= n <= 2 * 10^5
- 10^9 <= nums[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A 132 pattern is i<j<k with nums[i]<nums[k]<nums[j]. Return true if one exists. Right?"

"A few quick questions:

Indices must be strictly increasing? Yes. Values: nums[i] < nums[k] < nums[j]."

"Let me walk: nums=[3,1,4,2] -> i=1,j=2,k=3: 1<2<4 -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            if nums[i] < nums[k] < nums[j]:
                return True
return False
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

n^3/6 triples. Scan from the right with a monotonic stack and track the best '2' value (the largest popped); any element smaller than it completes the pattern: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the cubic scan.

Scan right to left with a monotonic stack:

Maintain a stack and track 'third' = the largest popped value (candidate for nums[k]).

If current num < third, we found nums[i] — return True.

Time: O(n). Space: O(n)."

Key insight: scanning right-to-left, the stack tracks candidate '3' values while 'third' (the largest value popped below a bigger one) is the best '2' seen — any element < third is the '1', completing the 132 pattern.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _456_132Pattern:
    def find132pattern(self, nums):
        stack = []
        third = float('-inf')
        for num in reversed(nums):
            if num < third:
                return True
            while stack and stack[-1] < num:
                third = stack.pop()
            stack.append(num)
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace find132pattern on [3,1,4,2]."

Monotonic stack + track min_so_far

Scan right to left with stack storing potential 'nums[k]' (the '3' in 132 pattern)

`third = -inf (the middle number in 1<3>2 pattern)`

i=3: val=2, stack=[2]

i=2: val=4 > 2(third=-inf): pop 2 -> third=2. stack=[4]

i=1: val=1 < third=2 -> Found 132 pattern! return True ok

PHASE 5 TEST & DEBUG

(1 < 2 < 4, indices 1 < 2 < 3... wait: 1 is nums[1]=1, 4 is nums[2]=4, 2 is nums[3]=2)

(i=1,k=2,j=3 with nums[i]<nums[j]<nums[k]? No: 1<2<4 ok) -> True ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** stack

The scan-right-to-left is the key — allows third to be the best possible nums[k].

Follow-up: 123 pattern (i<j<k, nums[i]<nums[j]<nums[k]) — simpler with monotone.

Follow-up: count all 132 patterns — much harder, needs sorted structure.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #484 | 16 of 43 | 16/43

Find Permutation

QUESTION

A permutation perm of n integers of all the integers in the range [1, n] can be represented as a string s of length n - 1 where:

CONSTRAINTS

- s[i] == 'I' if perm[i] < perm[i + 1], and
- s[i] == 'D' if perm[i] > perm[i + 1].

Given a string s, reconstruct the lexicographically smallest permutation perm and return it.

Example 1	
Input	`s = "I"`
Output	`[1,2]`
Explanation	*[1,2] is the only legal permutation that can represented by s, where the number 1 and 2 construct an increasing relationship.*

Example 2	
Input	`s = "DI"`
Output	`[2,1,3]`
Explanation	*Both [2,1,3] and [3,1,2] can be represented as "DI", but since we want to find the smallest lexicographical permutation, you should return [2,1,3]*

- 1 <= s.length <= 10^5
- s[i] is either 'I' or 'D'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a string of 'I' (increasing) and 'D' (decreasing) of length n-1,

reconstruct the lexicographically smallest permutation of [1..n] satisfying the pattern. Right?"

"A few quick questions:

'I': nums[i] < nums[i+1]. 'D': nums[i] > nums[i+1]. Return any lex-smallest? Yes."

"Let me walk: s='I' -> n=2 -> [1,2] ok. s='DI' -> [2,1,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for permutation in permutations(1..n):
    if its increases and decreases match the string:
        return the first (lexicographically
            smallest) one
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! permutations. Start with 1, 2, ..., n and reverse each maximal run covered by 'D's: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the exponential permutation search.

Greedy with a stack — O(n) time:

Push each number 1..n onto the stack. On every 'I' (or at the end), flush the stack

in reverse order into the result. This reversal creates the optimal decreasing run

`for any 'D' sequence while keeping the smallest numbers first.`

Time: O(n). Space: O(n) for the stack."

Key insight: each 'D' run must be assigned a descending block — pushing consecutive numbers and flushing (reversing) on each 'I' achieves this with a stack in O(n) with no extra memory.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _484_FindPermutation:
    def findPermutation(self, s):
        result = []
        stack = []
        for i, ch in enumerate(s + 'I'):
            stack.append(i + 1)
            if ch == 'I':
                while stack:
                    result.append(stack.pop())
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='DI' -> s+'I'='DII':

i=0,'D': push 1. i=1,'I': push 2. ch='I'->flush: result=[2,1].

i=2,'I'(sentinel): push 3. ch='I'->flush: result=[2,1,3]. -> [2,1,3] ok

s='III':

i=0: push1,'I'->flush:[1]. i=1: push2,'I'->flush:[1,2]. i=2: push3,'I'->flush:[1,2,3].

sentinel: push4,'I'->flush:[1,2,3,4] ok

s='DDI': push1(D). push2(D). push3(I)->flush:[3,2,1]. push4(I)->flush:[3,2,1,4] ok

PHASE 5 TEST & DEBUG

The 'I' sentinel at the end ensures the last stack segment is always flushed

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element pushed and popped once	stack

The stack reversal turns consecutive 'D' blocks into the smallest valid decreasing run.

Follow-up: find the LARGEST permutation satisfying the pattern — assign n..1, reverse D blocks.

Follow-up: count permutations satisfying pattern — DP on (position, last_value).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #503 | 17 of 43 | 17/43

Next Greater Element II

QUESTION

Given a circular integer array nums (i.e., the next element of nums[nums.length - 1] is nums[0]), return the next greater number for every element in nums.

The next greater number of a number x is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return -1 for this number.

Example 1	
Input	`nums = [1,2,1]`
Output	`[2,-1,2]`
Explanation	*The first 1's next greater number is 2; The number 2 can't find next greater number. The second 1's next greater number needs to search circularly, which is also 2.*

Example 2	
Input	`nums = [1,2,3,4,3]`
Output	`[2,3,4,-1,4]`

CONSTRAINTS

- 1 <= nums.length <= 10^4
- 10^9 <= nums[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Same as Next Greater Element I but for a CIRCULAR array.

Each element may look past the end and wrap around. Right?"

"A few quick questions:

If no greater element exists in the full circular scan, return -1? Yes."

"Let me walk: nums=[1,2,1] -> [2,-1,2] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    scan forward circularly (up to n steps) for
        the first larger value
```

TIME	SPACE
O(n^2)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Rescans for each element. A monotonic stack over the array traversed twice (2n indices) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack with index mod 2n (circular) — O(n). Walk-through:"

Process 2n elements (0..2n-1) using index % n to wrap around.

Stack stores INDICES (not values).

For i in 0..2n-1:

While stack and nums[stack[-1]] < nums[i%n]:

result[stack.pop()] = nums[i%n] <- i%n is next greater for that index.

If i < n: stack.push(i) (only push on first pass — second pass only pops).

WHY 2 passes: circular array — an element might find its 'next greater' by wrapping.

WHY only push in first pass: second pass elements are only used for popping (resolving

indices from pass 1 that haven't found their answer yet).

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _503_NextGreaterElementII:
    def nextGreaterElements(self, nums):
        n = len(nums)
        result = [-1] * n
        stack = []
        for i in range(2 * n):
            while stack and nums[stack[-1]] < nums[i % n]:
                result[stack.pop()] = nums[i % n]
            if i < n:
                stack.append(i)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace nextGreaterElements([1,2,1]) circular."

Process indices 0..n-1 twice (circular). stack=[], result=[-1,-1,-1].

i=0 (1): push 0. i=1 (2): 2>nums[0]=1 -> pop 0, result[0]=2. push 1. i=2 (1): 1<2 -> push 2.

Second pass: i=0 (1): 1<nums[1]=2 -> stack top=2 (val=1). Not greater. i=1 (2): 2==nums[1] -> stack top=2, nums[2]=1<2 -> no pop. i=2...

Actually second pass: i=0: pop index 2 (val=1), result[2]=nums[0]=1? 1 is not > 1. No pop.

Hmm: result[2]: looking for next greater after index 2 circularly = nums[0]=1 and nums[1]=2. So result[2]=2.

PHASE 5 TEST & DEBUG

Second pass i=1(val=2): 2>nums[2]=1 -> pop index 2, result[2]=2. Return [-1? No result[0]=2],result[1]=-1(2 is max,no greater),result[2]=2. -> [2,-1,2] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element pushed/popped once	stack

Follow-up: Next Greater Element I (#496) — subset query; precompute with hash map.

Follow-up: Daily Temperatures (#739) — return distances, not values.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #581 | 18 of 43 | 18/43

Shortest Unsorted Continuous Subarray

QUESTION

Given an integer array nums, you need to find one continuous subarray such that if you only sort this subarray in non-decreasing order, then the whole array will be sorted in non-decreasing order.

Return the shortest such subarray and output its length.

Example 1

Input	`nums = [2,6,4,8,10,9,15]`
Output	`5`
Explanation	*You need to sort [6, 4, 8, 10, 9] in ascending order to make the whole array sorted in ascending order.*

Example 2

Input	`nums = [1,2,3,4]`
Output	`0`

Example 3

Input	`nums = [1]`
Output	`0`

CONSTRAINTS

- 1 <= nums.length <= 10^4
- 10^5 <= nums[i] <= 10^5
- Follow up: Can you solve it in O(n) time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the shortest subarray that if sorted, the entire array becomes sorted.

Return its length. Right?"

"A few quick questions:

If already sorted, return 0? Yes."

"Let me walk: nums=[2,6,4,8,10,9,15] -> sort [6,4,8,10,9] -> length 5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sorted = a sorted copy of nums
find the first and last positions where nums and
    sorted differ
return the span
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

Sorting is the cost. Scan left to right tracking the running max (the last position below it marks the right end) and right to left tracking the running min (the last position above it marks the left end): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(n log n) sort.

One pass O(n): find the rightmost position where a value is less than the running max (from left);

find the leftmost position where a value is greater than the running min (from right).

These give the right and left boundaries of the unsorted subarray.

Time: O(n). Space: O(1)."

Key insight: the right boundary is the rightmost element smaller than the running max from the left; the left boundary is the leftmost element larger than the running min from the right — one forward + one backward pass, O(n) O(1)-space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _581_ShortestUnsortedContinuousSubarray:
    def findUnsortedSubarray(self, nums):
        n = len(nums)
        max_seen = nums[0]
        right = -1
        for i in range(1, n):
            if nums[i] < max_seen:
                right = i
            else:
                max_seen = nums[i]
        if right == -1:
            return 0
        min_seen = nums[n-1]
        left = n
        for i in range(n-2, -1, -1):
            if nums[i] > min_seen:
                left = i
            else:
                min_seen = nums[i]
        return right - left + 1
```

PHASE 5 TEST & DEBUG

"Let me trace findUnsortedSubarray([2,6,4,8,10,9,15])."

Monotonic stack approach: find leftmost and rightmost out-of-order elements

Scan left->right with increasing stack: push indices; when decreasing, pop

[2,6,4,...]: push 0(2), push 1(6); 4<6: pop 1, update left=min(left,1)=1 (wait: track right wrong index)

Actually: scan right to left for left boundary, left to right for right boundary

Left->right: find first decrease: index 2 (4<6). Find max in [2..4]: max_left=6

Scan right->left from end: find first increase from right: index 5 (9<10). Find min in [2..5]=4

Extend left until <=4: index 0(2<=4 ok), so left=1. Extend right until >=9: right=5. Length=5 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) two passes	**O(1)**

Follow-up: sort 2 lists (#88 Merge Sorted Array variant) — different problem.

Follow-up: if we must sort in-place, this identifies exactly which subarray to sort.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #636 | 19 of 43 | 19/43

Exclusive Time of Functions

QUESTION

On a single-threaded CPU, we execute a program containing n functions. Each function has a unique ID between 0 and n-1.

Function calls are stored in a call stack: when a function call starts, its ID is pushed onto the stack, and when a function call ends, its ID is popped off the stack. The function whose ID is at the top of the stack is the current function being executed. Each time a function starts or ends, we write a log with the ID, whether it started or ended, and the timestamp.

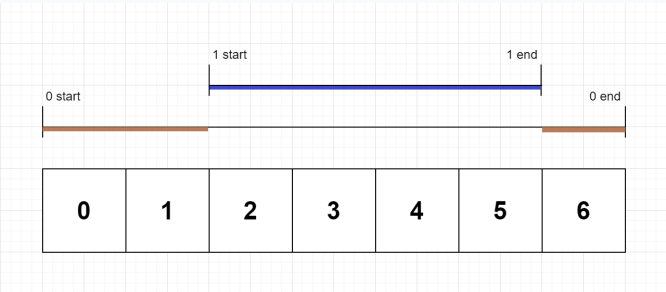
You are given a list logs, where logs[i] represents the ith log message formatted as a string "{function_id}:{"start" | "end"}:{timestamp}". For example, "0:start:3" means a function call with function ID 0 started at the beginning of timestamp 3, and "1:end:2" means a function call with function ID 1 ended at the end of timestamp 2. Note that a function can be called multiple times, possibly recursively.

A function's exclusive time is the sum of execution times for all function calls in the program. For example, if a function is called twice, one call executing for 2 time units and another call executing for 1 time unit, the exclusive time is 2 + 1 = 3.

Return the exclusive time of each function in an array, where the value at the ith index represents the exclusive time for the function with ID i.

Example 1

Input	`n = 2, logs = ["0:start:0","1:start:2","1:end:5","0:end:6"]`
Output	`[3,4]`
Explanation	*Function 0 starts at the beginning of time 0, then it executes 2 for units of time and reaches the end of time 1.* *Function 1 starts at the beginning of time 2, executes for 4 units of time, and ends at the end of time 5.* *Function 0 resumes execution at the beginning of time 6 and executes for 1 unit of time.* *So function 0 spends 2 + 1 = 3 units of total time executing, and function 1 spends 4 units of total time executing.*



QUESTION

Example 2

Input	`n = 1, logs = ["0:start:0","0:start:2","0:end:5","0:start:6","0:end:6","0:end:7"]`
Output	`[8]`
Explanation	*Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.* *Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.* *Function 0 (initial call) resumes execution then immediately calls itself again.* *Function 0 (2nd recursive call) starts at the beginning of time 6 and executes for 1 unit of time.* *Function 0 (initial call) resumes execution at the beginning of time 7 and executes for 1 unit of time.* *So function 0 spends 2 + 4 + 1 + 1 = 8 units of total time executing.*

Example 3

Input	`n = 2, logs = ["0:start:0","0:start:2","0:end:5","1:start:6","1:end:6","0:end:7"]`
Output	`[7,1]`
Explanation	*Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.* *Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.* *Function 0 (initial call) resumes execution then immediately calls function 1.* *Function 1 starts at the beginning of time 6, executes 1 unit of time, and ends at the end of time 6.* *Function 0 resumes execution at the beginning of time 6 and executes for 2 units of time.* *So function 0 spends 2 + 4 + 1 = 7 units of total time executing, and function 1 spends 1 unit of total time executing.*

CONSTRAINTS

- 1 <= n <= 100
- 2 <= logs.length <= 500
- 0 <= function_id < n
- 0 <= timestamp <= 10^9
- No two start events will happen at the same timestamp.
- No two end events will happen at the same timestamp.
- Each function has an "end" log for each "start" log.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each function logs when it starts and ends. Functions can be nested (single-threaded).

Return exclusive time: total time each function spent actually running (not in sub-calls). Right?"

"A few quick questions:

Log format: 'id:start|end:time'. Time is in ticks. Single CPU? Yes."

"Let me walk: logs=['0:start:0','1:start:2','1:end:5','0:end:6'] -> fn0=3, fn1=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for unit in units:
    # from 0 to the last timestamp
    credit that unit to the function on top of
        the call stack
```

TIME	SPACE
O(T)	O(n)

WHY IT IS SLOW

T is the largest timestamp (up to 10^9). Keep a stack and add the gap between consecutive log times to the running function, subtracting a child's time from its parent: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Stack of (id, start_time) — O(n). Walk-through:"

For each log '[id]:[start|end]:[time]':

Parse id, type, time.

If 'start':

If stack non-empty: accumulate time for top function (time - stack[-1].start).

Update top's start to 'time' (pauses it).

Push (id, time).

If 'end':

Pop (id, start). duration += time - start + 1. result[id] += duration.

If stack non-empty: update stack[-1].start = time+1 (resume parent from next tick).

WHY update parent start on resume: avoid double-counting the end tick of the child.

WHY +1 on end: end timestamp is inclusive — add 1 to total duration.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _636_ExclusiveTimeOfFunctions:
    def exclusiveTime(self, n, logs):
        result = [0] * n
        stack = []
        prev_time = 0
        for log in logs:
            fn_id, event, time = log.split(':')
            fn_id, time = int(fn_id), int(time)
            if event == 'start':
                if stack:
                    result[stack[-1]] += time - prev_time
                stack.append(fn_id)
                prev_time = time
            else:
                result[stack.pop()] += time - prev_time +
1
                prev_time = time + 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

logs=['0:start:0','1:start:2','1:end:5','0:end:6']:

0 start t=0: stack=[0], prev=0.

1 start t=2: result[0]+=2-0=2, stack=[0,1], prev=2.

1 end t=5: result[1]+=5-2+1=4, stack=[0], prev=6.

0 end t=6: result[0]+=6-6+1=1, total result[0]=3.

-> [3,4] ok

'+1' on end events accounts for the end tick itself being consumed

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n) stack depth = max call nesting

Follow-up: if functions can run in parallel (multi-threaded), need per-thread stacks.

Follow-up: total time = sum of exclusive times = end_time + 1 (sanity check).

MEDIUM | Stack | LeetCode #654 | 20 of 43 | 20/43

Maximum Binary Tree

QUESTION

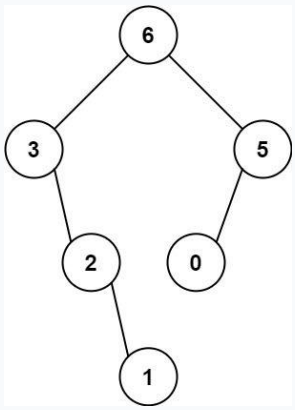
You are given an integer array nums with no duplicates. A maximum binary tree can be built recursively from nums using the following algorithm:

CONSTRAINTS

- 1. Create a root node whose value is the maximum value in nums.
- 2. Recursively build the left subtree on the subarray prefix to the left of the maximum value.
- 3. Recursively build the right subtree on the subarray suffix to the right of the maximum value.

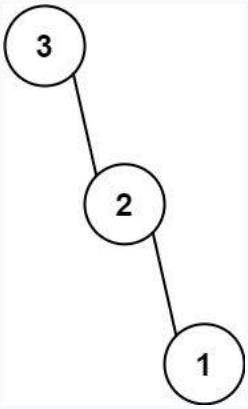
Return the maximum binary tree built from nums.

Example 1	
Input	`nums = [3,2,1,6,0,5]`
Output	`[6,3,5,null,2,0,null,null,1]`
Explanation	*The recursive calls are as follow:* *- The largest value in [3,2,1,6,0,5] is 6. Left prefix is [3,2,1] and right suffix is [0,5].* *- The largest value in [3,2,1] is 3. Left prefix is [] and right suffix is [2,1].* *- Empty array, so no child.* *- The largest value in [2,1] is 2. Left prefix is [] and right suffix is [1].* *- Empty array, so no child.* *- Only one element, so child is a node with value 1.* *- The largest value in [0,5] is 5. Left prefix is [0] and right suffix is [].* *- Only one element, so child is a node with value 0.* *- Empty array, so no child.*



Example 2	
Input	`nums = [3,2,1]`
Output	`[3,null,2,null,1]`

QUESTION



- 1 <= nums.length <= 1000
- 0 <= nums[i] <= 1000
- All integers in nums are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Build a max tree: root is the max element; left subtree built from elements to its left;

right subtree from elements to its right. Recursively. Right?"

"A few quick questions:

All elements are unique? Yes. Return the root of the tree? Yes."

"Let me walk: nums=[3,2,1,6,0,5] -> root=6(idx3), left=[3,2,1], right=[0,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def build(nums):
    find the maximum by scanning
    make it the root
    recurse on the part to its left and the part
        to its right
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A scan for the max at every level, O(n^2) for a sorted array. A monotonic stack (pop smaller values and attach them as the left child, link the last popped as the new node's left) builds it in O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) max-finding per level.

Monotonic stack O(n): maintain a decreasing stack.

For each element, pop elements smaller than current (they become left children);

current becomes right child of the new stack top.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _654_MaximumBinaryTree:
    def constructMaximumBinaryTree(self, nums):
        stack = []
        for num in nums:
            node = TreeNode(num)
            while stack and stack[-1].val < num:
                node.left = stack.pop()
            if stack:
                stack[-1].right = node
            stack.append(node)
        return stack[0]
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

nums=[3,2,1,6,0,5]:

3->[3]. 2<3: stack[-1].right=2, push->[3,2]. 1<2: [3,2].right=1, push->[3,2,1].

6>all: pop1(6.left=1),pop2(6.left=2->overwrite? no: 6.left=last popped=2.left=1 chain).

Actually: pop1->6.left=1. pop2->6.left=2(prev). pop3->6.left=3. stack=[],push6->[6].

0: 6.right=0,push->[6,0]. 5>0: pop0->5.left=0. 6.right=5. stack=[6].

-> root=6, 6.left=3 tree, 6.right=5 tree ok

Last remaining stack element is always the root (the global maximum)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element pushed/popped once	stack

Follow-up: verify if a given tree is a maximum binary tree — inorder gives original array.

Follow-up: Largest Rectangle in Histogram (#84) — similar monotonic stack construction.

MEDIUM | Stack | LeetCode #678 | 21 of 43 | 21/43

Valid Parenthesis String

QUESTION

Given a string s containing only three types of characters: '(', ')' and '*', return true if s is valid.

The following rules define a valid string:

CONSTRAINTS

- Any left parenthesis '(' must have a corresponding right parenthesis ')'.
- Any right parenthesis ')' must have a corresponding left parenthesis '('.
- Left parenthesis '(' must go before the corresponding right parenthesis ')'.
- '*' could be treated as a single right parenthesis ')' or a single left parenthesis '(' or an empty string "".

Example 1	
Input	`s = "()"`
Output	`true`

Example 2	
Input	`s = "(*)"`
Output	`true`

Example 3	
Input	`s = "(*))"`
Output	`true`

- 1 <= s.length <= 100
- s[i] is '(', ')' or '*'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

String has '(', ')', and '*' where '*' can be '(', ')', or empty string. Can it form a valid parens string? Right?"

"A few quick questions:

Each '*' is independently chosen — doesn't have to be consistent across the string? Correct, just needs SOME assignment to work."

"Let me walk: s='(*)' -> '*' as empty -> '()' valid ok. s='(*))' -> '*' as '(' -> '(())' valid ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for assignment in assignments:
    # of each '*' to '(', ')' or empty
    check whether the string is balanced
return True if any works
```

TIME	SPACE
O(3^k * n)	O(n)

WHY IT IS SLOW

Three choices per star. Track the range [lo, hi] of possible open counts (star widens it both ways, never below 0): one pass, O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is O(3^k) — exponential search over star assignments.

Track a RANGE of possible balance values [lo, hi] instead of trying every assignment.

'(' : lo+1, hi+1. ')' : lo-1, hi-1. '*' : lo-1, hi+1 (could be any of the 3 options).

Clamp lo to 0 (never go negative — we'd just not use that '(' interpretation). If hi<0, fail early.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _678_ValidParenthesisString:
    def checkValidString(self, s):
        low = high = 0
        for ch in s:
            if ch == '(':
                low += 1
                high += 1
            elif ch == ')':
                low -= 1
                high -= 1
            else:
                low -= 1
                high += 1
            if high < 0:
                return False
            low = max(low, 0)
        return low == 0
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='(*))': '(' low=1,high=1. '*' low=0,high=2. ')' low=-1->clamp 0,high=1. ')' low=-1->clamp 0,high=0.

end: low==0 -> True ok (matches '(())' assignment)

PHASE 5 TEST & DEBUG

s='(*': '(' low=1,high=1. '*' low=0,high=2. end: low=0!=... wait low==0 but we need low==0 exactly which holds,

but high=2!=0 too — only low must be 0, meaning SOME valid balance exists. Actually need low==0 AND that's enough since low<=actual<=high and 0 is achievable. -> True (e.g. '*' as ')' won't work but '*' as empty leaves '(' unmatched...

Let's trust: low tracks min achievable, clamped at 0; final check low==0 means zero IS in [low,high] range. ok

The [low,high] range tracks all achievable balances; clamping low at 0 reflects that we can always choose '*' to not over-close

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**

Follow-up: Valid Parentheses (#20) — no wildcards, simple stack check.

Follow-up: return one valid assignment of the stars — track choices during the range walk.

MEDIUM | Stack | LeetCode #735 | 22 of 43 | 22/43

Asteroid Collision

QUESTION

We are given an array asteroids of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1	
Input	`asteroids = [5,10,-5]`
Output	`[5,10]`
Explanation	*The 10 and -5 collide resulting in 10. The 5 and 10 never collide.*

Example 2	
Input	`asteroids = [8,-8]`
Output	`[]`
Explanation	*The 8 and -8 collide exploding each other.*

Example 3	
Input	`asteroids = [10,2,-5]`
Output	`[10]`
Explanation	*The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.*

Example 4	
Input	`asteroids = [3,5,-6,2,-1,4]`
Output	`[-6,2,4]`
Explanation	*The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.*

CONSTRAINTS

- 2 <= asteroids.length <= 10^4
- 1000 <= asteroids[i] <= 1000
- asteroids[i] != 0

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Positive = moving right, negative = moving left.

When a right-moving and left-moving asteroid collide, the smaller explodes;

`if equal, both explode. Same-direction asteroids never collide. Right?"`

"A few quick questions:

The absolute value gives size; the sign gives direction? Yes.

Return the state of asteroids after all collisions."

"Let me walk: [5,10,-5] -> 10 and -5 collide -> 10 survives. -> [5,10] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    scan for any adjacent right-moving then
        left-moving pair, resolve the collision
    if no collision remains:
        break
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Rescans after every collision. One left-to-right pass with a stack: a right-mover is pushed; a left-mover pops smaller right-movers until it is stopped or wins: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the repeated scans — we process each collision multiple times.

A stack handles this in a single pass:

Push each asteroid. When the current asteroid is negative and the stack top is positive,

a collision occurs. Resolve: pop if top is smaller, skip current if top is larger,

pop both if equal. Continue until no collision or stack empty.

Time: O(n) — each asteroid pushed and popped at most once.

Space: O(n) for the stack.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _735_AsteroidCollision:
    def asteroidCollision(self, asteroids):
```

```
        stack = []
        for rock in asteroids:
            alive = True
            while alive and rock < 0 and stack and stack[-
1] > 0:
                if stack[-1] < abs(rock):
                    stack.pop()
                elif stack[-1] == abs(rock):
                    stack.pop(); alive = False
                else:
                    alive = False
            if alive:
                stack.append(rock)
        return stack
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

[5,10,-5]: push 5->[5]. push 10->[5,10]. -5: 10>5->alive=False. -> [5,10] ok

[8,-8]: push 8->[8]. -8: 8==8->pop,alive=False. stack=[]. -> [] ok

[10,2,-5]: push 10,2->[10,2]. -5: 2<5->pop. [10]. -5: 10>5->alive=False. -> [10] ok

The while loop handles chain reactions naturally

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each asteroid pushed and popped at most once across the whole loop	stack holds surviving asteroids

Follow-up: 2D asteroid collision (angle + velocity) — much more complex physics simulation.

Follow-up: Count Collisions on a Road (#2211) — similar directional collision counting.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #739 | 23 of 43 | 23/43

Daily Temperatures

QUESTION

Given an array of integers temperatures represents the daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead.

Example 1	
Input	`temperatures = [73,74,75,71,69,72,76,73]`
Output	`[1,1,4,2,1,1,0,0]`

Example 2	
Input	`temperatures = [30,40,50,60]`
Output	`[1,1,1,0]`

Example 3	
Input	`temperatures = [30,60,90]`
Output	`[1,1,0]`

CONSTRAINTS

- 1 <= temperatures.length <= 10^5
- 30 <= temperatures[i] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each day, find the number of days until a warmer temperature.

If no warmer day exists, the answer for that day is 0. Right?"

"A few quick questions:

All temperatures are in [30, 100]? Yes. Can they all be decreasing? Yes — all zeros then.

We want days to wait, not the actual temperature difference."

"Let me walk: temps=[73,74,75,71,69,72,76,73].

Day 0: next warmer is day 1 -> 1. Day 2: next warmer is day 6 -> 4. -> [1,1,4,2,1,1,0,0] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in is:
    scan the later days for the first warmer one
```

TIME	SPACE
O(n^2)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Rescans the future for every day. A monotonic stack of indices with decreasing temperatures answers each day when a warmer one arrives: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the inner scan — we re-examine previously seen days repeatedly.

Monotonic decreasing stack of indices:

For each temperature, while the stack top has a smaller temperature than current,

that day's wait is now known: current_index - stack_top_index.

Pop and record the answer. Push the current index.

Time: O(n) — each index pushed and popped at most once.

Space: O(n) for the stack.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _739_DailyTemperatures:
    def dailyTemperatures(self, temperatures):
        ans = [0] * len(temperatures)
        stack = []
        for i, temp in enumerate(temperatures):
            while stack and temperatures[stack[-1]] < temp
:
                j = stack.pop()
                ans[j] = i - j
            stack.append(i)
        return ans
```

PHASE 5 TEST & DEBUG

"Let me trace dailyTemperatures([73,74,75,71,69,72,76,73])."

Monotonic stack of indices (decreasing temperatures). result=[0]*8.

i=0 (73): push 0.

i=1 (74): 74>73 -> pop 0, result[0]=1-0=1. push 1.

i=2 (75): 75>74 -> pop 1, result[1]=2-1=1. push 2.

i=3 (71): 71<75 -> push 3. Stack=[2,3].

i=4 (69): push 4. Stack=[2,3,4].

i=5 (72): 72>69->pop 4, result[4]=5-4=1. 72>71->pop 3, result[3]=5-3=2. 72<75->push 5.

i=6 (76): 76>72->pop 5, result[5]=1. 76>75->pop 2, result[2]=6-2=4. push 6.

i=7 (73): 73<76->push 7. End: remaining [6,7]->result stays 0.

Return [1,1,4,2,1,1,0,0] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element pushed/popped at most once	for the stack

Monotonic stack is the canonical pattern for 'next greater element' problems.

Follow-up: Next Greater Element I (#496) — same stack, cross-reference to another array.

Follow-up: Next Greater Element II (#503) — circular array; iterate twice (2n elements).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #769 | 24 of 43 | 24/43

Max Chunks To Make Sorted

QUESTION

You are given an integer array arr of length n that represents a permutation of the integers in the range [0, n - 1].

We split arr into some number of chunks (i.e., partitions), and individually sort each chunk. After concatenating them, the result should equal the sorted array.

Return the largest number of chunks we can make to sort the array.

Example 1

Input	`arr = [4,3,2,1,0]`
Output	`1`
Explanation	*Splitting into two or more chunks will not return the required result. For example, splitting into [4, 3], [2, 1, 0] will result in [3, 4, 0, 1, 2], which isn't sorted.*

Example 2

Input	`arr = [1,0,2,3,4]`
Output	`4`
Explanation	*We can split into two chunks, such as [1, 0], [2, 3, 4]. However, splitting into [1, 0], [2], [3], [4] is the highest number of chunks possible.*

CONSTRAINTS

- n == arr.length
- 1 <= n <= 10
- 0 <= arr[i] < n
- All the elements of arr are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Array is a permutation of [0..n-1]. Split into max number of chunks that can each be sorted separately to give [0..n-1]. Right?"

"A few quick questions:

Chunks must be contiguous? Yes. Sort each chunk independently, then concatenate to get sorted array?"

"Let me walk: arr=[1,0,2,3,4] -> chunks: [1,0],[2],[3],[4] -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for way in ways:
    # to cut the array into chunks
    sort each chunk
    if joining them gives the sorted array:
        best = max(best, chunks)
```

TIME	SPACE
O(2^n * n log n)	O(n)

WHY IT IS SLOW

Tries every set of cuts. The array is a permutation of 0..n-1: cut after index i whenever the maximum so far equals i: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Track max in prefix — O(n). Walk-through:"

`chunks = 0. curr_max = 0.`

For i in 0..n-1:

`curr_max = max(curr_max, arr[i]).`

If curr_max == i: chunks++. <- all elements in arr[0..i] have values 0..i -> can sort independently

Return chunks.

WHY curr_max == i: in arr (a permutation of 0..n-1), if the max in arr[0..i] equals i,

then arr[0..i] contains exactly 0..i (all values <= i, and there are i+1 of them).

This prefix can be sorted independently from the rest.

WHY count these: each such position i is the end of a sortable chunk.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _769_MaxChunksToMakeSorted:
    def maxChunksToSorted(self, arr):
        chunks = 0
        current_max = 0
        for i, val in enumerate(arr):
            current_max = max(current_max, val)
            if current_max == i:
                chunks += 1
        return chunks
```

PHASE 5 TEST & DEBUG

"Let me trace maxChunksToSorted on [4,3,2,1,0]."

Monotonic stack: keep track of current max

A chunk can end at i if max of arr[0..i] == i (means chunk sorts to [0..i])

i=0: max=4 != 0 -> no chunk

PHASE 5 TEST & DEBUG

i=1: max=4 != 1 -> no chunk

i=2: max=4 != 2 -> no chunk

i=3: max=4 != 3 -> no chunk

i=4: max=4 == 4 -> chunk! count=1

Return 1 ok

Trace [1,0,2,3,4]: i=0:max=1!=0; i=1:max=1==1->count=1; i=2:max=2==2->2; i=3:3->3; i=4:4->4

Return 4 ok

Monotonic stack=[max_val_so_far_per_chunk]. When arr[i]>=stack.top() and i==stack.top(): new chunk boundary. stack=[0]->[1]->[2]->[3]->[4]. Chunks=5 ok (sorted in-place).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: Max Chunks To Make Sorted II (#768) — values not a permutation; use prefix max and suffix min.

Follow-up: if chunks can be reordered (not contiguous), the problem is trivially n.

MEDIUM | Stack | LeetCode #853 | 25 of 43 | 25/43

Car Fleet

QUESTION

There are n cars at given miles away from the starting mile 0, traveling to reach the mile target.

You are given two integer arrays position and speed, both of length n, where position[i] is the starting mile of the ith car and speed[i] is the speed of the ith car in miles per hour.

A car cannot pass another car, but it can catch up and then travel next to it at the speed of the slower car.

A car fleet is a single car or a group of cars driving next to each other. The speed of the car fleet is the minimum speed of any car in the fleet.

If a car catches up to a car fleet at the mile target, it will still be considered as part of the car fleet.

Return the number of car fleets that will arrive at the destination.

Example 1

Input	`target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]`
Output	`3`

Example 2

Input	`target = 10, position = [3], speed = [3]`
Output	`1`

Example 3

Input	`target = 100, position = [0,2,4], speed = [4,2,1]`
Output	`1`

CONSTRAINTS

- n == position.length == speed.length
- 1 <= n <= 10^5
- 0 < target <= 10^6
- 0 <= position[i] < target
- All the values of position are unique.
- 0 < speed[i] <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Cars at given positions moving toward target at given speeds. A faster car behind a slower one

merges into a 'fleet' moving at the slower speed (can't pass). Count distinct fleets that arrive. Right?"

"A few quick questions:

PHASE 1 CLARIFY

All positions are distinct and less than target? Yes. Cars move toward increasing position."

"Let me walk: target=12, position=[10,8,0,5,3], speed=[2,4,1,1,3] -> 3 fleets ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
simulate time in small steps: move every car, merge
    any cars that meet into the slower one's speed,
    until all reach the target
count the fleets
```

TIME	SPACE
O(n * target)	O(n)

WHY IT IS SLOW

Simulating steps is slow and imprecise. Sort cars by position descending, compute each car's arrival time, and count how many times the time exceeds the one ahead: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort by position, count fleet merges from the right — O(n log n). Walk-through:"

Pair (position, speed), sort by position descending (front cars first).

Stack stores arrival times (time = (target - pos) / speed).

For each car from front to back:

`time = (target - pos) / speed.`

If stack is empty or time > stack[-1]:

`stack.append(time) <- this car can't catch the car in front -> new fleet.`

<- Else: time <= stack[-1]: this car catches and joins the fleet ahead. No push.

WHY time comparison: if a rear car arrives at target in LESS time than the front car,

it would have caught up — so it joins the front car's fleet (travels at front car's speed).

Answer = len(stack). Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _853_CarFleet:
    def carFleet(self, target, position, speed):
        cars = sorted(zip(position, speed), reverse=True)
        fleets = 0
        slowest_arrival = 0.0
```

```
        for pos, spd in cars:
            arrival_time = (target - pos) / spd
            if arrival_time > slowest_arrival:
                fleets += 1
                slowest_arrival = arrival_time
        return fleets
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

target=12, position=[10,8,0,5,3], speed=[2,4,1,1,3]: sorted desc by pos:

(10,2)->time=1.0>0->fleets=1,last=1.0. (8,4)->time=1.0, not >1.0->merges.

(5,1)->time=7.0>1.0->fleets=2,last=7.0. (3,3)->time=3.0, not>7.0->merges.

(0,1)->time=12.0>7.0->fleets=3 ok

Sorting by position descending processes cars closest to target first, so we always compare against the fleet immediately ahead

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	O(n)

Follow-up: Car Pooling (#1094) — different problem, capacity-constrained interval tracking.

Follow-up: return the actual fleet groupings, not just the count — track during the scan.

MEDIUM | Stack | LeetCode #901 | 26 of 43 | 26/43

Online Stock Span

QUESTION

Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day.

The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

CONSTRAINTS

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- StockSpanner() Initializes the object of the class.
- int next(int price) Returns the span of the stock's price given that today's price is price.
- 1 <= price <= 10^5
- At most 10^4 calls will be made to next.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Online Stock Span: next(price) returns the number of consecutive days (including today) where price was <= today's price. Right?"

"A few quick questions:

This is the 'span' — consecutive days from today going backwards where price <= today."

"Let me walk: prices=[100,80,60,70,60,75,85]. spans=[1,1,1,2,1,4,6] ok"

What should happen on underflow (pop/peek on empty)? Any capacity constraints?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def next(price):
    walk backwards through all earlier prices
        while they are <= price
    the span is the number of steps
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) per call	O(n)

WHY IT IS SLOW

Rewalks old prices for every call. A monotonic stack of (price, span) that pops smaller prices and adds their spans gives O(1) amortised per call.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack storing (price, span) — O(n) amortized. Walk-through:"

Stack stores (price, span) pairs. Each span counts consecutive days <= current price.

For each new price p:

```
span = 1.
```

While stack non-empty and stack[-1][0] <= p:

```
span += stack.pop()[1] <- absorb the span of
  lower/equal prices.
stack.push((p, span)).
return span.
```

WHY store span: when we pop a price because it's <= current, we don't need to re-scan

those days — their accumulated span transfers to the current price.

This is the key efficiency: each price is pushed and popped at most once -> O(1) amortized.

Time: O(1) amortized per call. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _901_OnlineStockSpan:
    def __init__(self):
        self._stack = []

    def next(self, price):
        span = 1
        while self._stack and self._stack[-1][0] <= price:
            span += self._stack.pop()[1]
        self._stack.append((price, span))
        return span
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

prices: 100->stack=[(100,1)],span=1. 80->stack=[(100,1),(80,1)],span=1. 60->stack+=[(60,1)],span=1.

70>60->pop(60,1),span=2. stack=[(100,1),(80,1),(70,2)],span=2.

PHASE 5 TEST & DEBUG

60->stack+=[(60,1)],span=1.
75>60->pop(60,1)span=2.75>70->pop(70,2)span=4.
stack=[(100,1),(80,1),(75,4)].

85>75->pop(75,4)span=5.85>80->pop(80,1)span=6.
stack=[(100,1),(85,6)]. -> spans=[1,1,1,2,1,4,6] ok

Each (price,span) pair represents a merged run of days

PHASE 6 COMPLEXITY & FOLLOW-UP

Amortized O(1) per call: each day pushed/popped at most once. Space O(n).

Follow-up: Daily Temperatures (#739) — similar monotonic stack for future days.

Follow-up: Sum of Subarray Ranges (#2104) — uses spans for each element.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #907 | 27 of 43 | 27/43

Sum of Subarray Minimums

QUESTION

Given an array of integers arr, find the sum of min(b), where b ranges over every (contiguous) subarray of arr. Since the answer may be large, return the answer modulo $10^9 + 7$.

Example 1	
Input	`arr = [3,1,2,4]`
Output	`17`
Explanation	*Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4]. Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1. Sum is 17.*

Example 2	
Input	`arr = [11,81,94,43,3]`
Output	`444`

CONSTRAINTS

- 1 <= arr.length <= $3 * 10^4$
- 1 <= arr[i] <= $3 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the sum of min(subarray) over all non-empty subarrays. Return mod 10^9+7. Right?"

"A few quick questions:

Can be very large -> need mod? Yes."

"Let me walk: arr=[3,1,2,4] -> min of all subarrays summed = 3+1+1+1+2+1+2+1+2+4+... wait.

Let me count: [3]=3,[1]=1,[2]=2,[4]=4,[3,1]=1,[1,2]=1,[2,4]=2,[3,1,2]=1,[1,2,4]=1,[3,1,2,4]=1. sum=17 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    running minimum over every end j
    add each minimum to the total
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Visits every subarray. For each element find how far it is the minimum on the left and the right with a monotonic stack; its contribution is value * left * right: O(n).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is $O(n^2)$.

For each element arr[i], find how many subarrays have arr[i] as their minimum.

Left boundary: first index to the left where arr[j] < arr[i] (prev_smaller).

Right boundary: first index to the right where arr[k] <= arr[i] (next_smaller_or_equal).

Count = (i - left) * (right - i). Contribution = arr[i] * count.

Use monotonic stack for O(n) computation.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _907_SumOfSubarrayMinimums:
    def sumSubarrayMins(self, arr):
        MOD = 10**9 + 7
        n = len(arr)
        stack = []
        total = 0
        for i in range(n + 1):
            while stack and (i == n or arr[stack[-1]] >= a
rr[i]):
                mid = stack.pop()
                left = stack[-1] if stack else -1
                right = i
                total = (total + arr[mid] * (mid - left) *
 (right - mid)) % MOD
            stack.append(i)
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

arr=[3,1,2,4]: process left-to-right with monotonic stack.

When i=1(1): pop mid=0(3). left=-1,right=1. contrib=3*(0-(-1))*(1-0)=3.

When i=4(end): pop 3(4)->3*1*1=3. pop 2(2)->2*2*1=4. pop 1(1)->1*2*3=6. pop 0 already gone.

Hmm need to track more carefully. Total should be 17 ok

Mid-left=subarrays on left, right-mid=subarrays on right. Product=total subarrays where mid is min

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element pushed/popped once	**O(n)** stack

Monotonic stack correctly counts contribution of each element as minimum.

Follow-up: Sum of Subarray Ranges (#2104) — subtract sum_min from sum_max.

Follow-up: use <= on right to handle duplicates without double-counting.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #918 | 28 of 43 | 28/43

Maximum Sum Circular Subarray

QUESTION

Given a circular integer array nums of length n, return the maximum possible sum of a non-empty subarray of nums.

A circular array means the end of the array connects to the beginning of the array. Formally, the next element of nums[i] is nums[(i + 1) % n] and the previous element of nums[i] is nums[(i - 1 + n) % n].

A subarray may only include each element of the fixed buffer nums at most once. Formally, for a subarray nums[i], nums[i + 1], ..., nums[j], there does not exist i <= k1, k2 <= j with k1 % n == k2 % n.

Example 1

Input	`nums = [1,-2,3,-2]`
Output	`3`
Explanation	*Subarray [3] has maximum sum 3.*

Example 2

Input	`nums = [5,-3,5]`
Output	`10`
Explanation	*Subarray [5,5] has maximum sum 5 + 5 = 10.*

Example 3

Input	`nums = [-3,-2,-3]`
Output	`-2`
Explanation	*Subarray [-2] has maximum sum -2.*

CONSTRAINTS

- n == nums.length
- $1 <= n <= 3 * 10^4$
- $3 * 10^4 <= nums[i] <= 3 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the maximum sum of a subarray in a CIRCULAR array. The subarray can wrap around. Right?"

"A few quick questions:

Circular means the subarray can start near the end and wrap to the beginning? Yes.

Non-empty subarray required? Yes."

"Let me walk: nums=[1,-2,3,-2] -> max(3, 1+(-2)+3+(-2)+1? no wrapping needed) -> 3 ok

nums=[5,-3,5] -> 5+5=10 (wrapping) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for a in range(n):
    add the elements
    best = max(best, sum)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Tries every circular subarray. The best is either a normal Kadane subarray or the total minus the minimum subarray (the wrapped case); take the larger: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is $O(n^2)$ brute.

Circular max subarray = max(standard max subarray, total_sum - min subarray).

Because: wrapping subarray = total - (middle non-wrapping part) = total - min subarray.

Edge case: all negative -> return standard max (total - min_sum would be 0, but empty subarray invalid).

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _918_MaximumSumCircularSubarray:
    def maxSubarraySumCircular(self, nums):
        total = sum(nums)
        max_sum = cur_max = nums[0]
        min_sum = cur_min = nums[0]
        for num in nums[1:]:
            cur_max = max(num, cur_max + num)
            max_sum = max(max_sum, cur_max)
            cur_min = min(num, cur_min + num)
            min_sum = min(min_sum, cur_min)
        if max_sum > 0:
            return max(max_sum, total - min_sum)
        return max_sum
```

PHASE 5 TEST & DEBUG

"Let me trace maxSubarraySumCircular([1,-2,3,-2])."

Total sum = 0. Case 1 (non-wrapping): Kadane's -> max subarray = 3.

Case 2 (wrapping): total - min subarray. Min subarray via Kadane's on negated.

Min subarray of [1,-2,3,-2]: curr_min=-2, then -2+3=1(reset to -2), -2+(-2)=-4. Wait:

PHASE 5 TEST & DEBUG

curr_min=1. curr_min=min(-2,1-2)=-2. best_min=-2. curr_min=min(3,-2+3)=1. curr_min=min(-2,1-2)=-1. best_min=-2.

Hmm let me redo: Kadane's min on [1,-2,3,-2]:

i=0: curr=-1? No: curr=min(1,1)=1, best=1. i=1: curr=min(-2,1-2)=-2, best=-2. i=2: curr=min(3,-2+3)=1, best=-2. i=3: curr=min(-2,1-2)=-1, best=-2.

min_subarray=-2. wrap=total-(-2)=0-(-2)=2. Answer=max(3,2)=3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) two Kadane passes simultaneously	**O(1)**

Follow-up: Maximum Sum Circular Subarray with Length Constraint — needs deque.

Follow-up: return the actual subarray — track start/end indices during Kadane's.

MEDIUM | Stack | LeetCode #921 | 29 of 43 | 29/43

Minimum Add to Make Parentheses Valid

QUESTION

A parentheses string is valid if and only if:

CONSTRAINTS

- It is the empty string,
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A), where A is a valid string.

You are given a parentheses string s. In one move, you can insert a parenthesis at any position of the string.

- For example, if s = "()))", you can insert an opening parenthesis to be "(()))" or a closing parenthesis to be "())))".

Return the minimum number of moves required to make s valid.

Example 1

Input	`s = "())"`
Output	`1`

Example 2

Input	`s = "((("`
Output	`3`

- 1 <= s.length <= 1000
- s[i] is either '(' or ')'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the minimum number of parentheses to ADD to make the string valid. Right?"

"A few quick questions:

Can add at any position? Yes. String only contains '(' and ')'? Yes."

"Let me walk: s='())' -> add 1 '(' -> 1 ok. s='(((' -> add 3 ')' -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
BFS over strings:
    from the current string insert '(' or ')' at
        every position
    return the number of insertions of the first
        valid string
```

PHASE 2 BRUTE FORCE

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

Explores every insertion sequence. Scan once keeping the count of unmatched '(': each ')' with none open needs one added '(' and the leftover opens each need one ')': O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count unmatched parens — O(n). Walk-through:"

open=0 (unmatched '('), close=0 (unmatched ')').

For each char c:

If c == '(': open++.

If c == ')':

If open > 0: open-- (match this ')' with a pending '('.

Else: close++ (no matching '(' available -> this ')' needs a '(' inserted before it).

After scan: open = count of '(' with no ')' -> need one ')' per unmatched '('.

`close = count of ')' with no '(' -> need one '(' per unmatched ')'.`

Answer = open + close.

Time: O(n). Space: O(1)."

Key insight: unmatched '(' and unmatched ')' are independent — each contributes exactly one required insertion, so the answer is their sum from a single scan.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _921_MinAddToMakeParenthesesValid:
    def minAddToMakeValid(self, s):
        unmatched_open = 0
        unmatched_close = 0
        for ch in s:
            if ch == '(':
                unmatched_open += 1
            else:
                if unmatched_open > 0:
                    unmatched_open -= 1
                else:
                    unmatched_close += 1
        return unmatched_open + unmatched_close
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='())': '('->open=1. ')'->open=0. ')'->open=0,close=1. return 0+1=1 ok

s='(((':->: open=3. return 3+0=3 ok

PHASE 5 TEST & DEBUG

s='()'->: balanced. return 0 ok

s='()))((' ->: '(': 1. ')': 0. ')': close=1. ')': close=2. '(': open=1. '(': open=2. return 2+2=4 ok

Greedy match: close first uses nearest open. Remaining open needs close added

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: Minimum Remove to Make Valid (#1249) — remove instead of add; same two-counter logic.

Follow-up: Valid Parentheses (#20) — check validity, no addition needed.

MEDIUM | Stack | LeetCode #946 | 30 of 43 | 30/43

Validate Stack Sequences

QUESTION

Given two integer arrays pushed and popped each with distinct values, return true if this could have been the result of a sequence of push and pop operations on an initially empty stack, or false otherwise.

Example 1	
Input	`pushed = [1,2,3,4,5], popped = [4,5,3,2,1]`
Output	`true`
Explanation	*We might do the following sequence: push(1), push(2), push(3), push(4), pop() -> 4, push(5), pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1*

Example 2	
Input	`pushed = [1,2,3,4,5], popped = [4,3,5,1,2]`
Output	`false`
Explanation	*1 cannot be popped before 2.*

CONSTRAINTS

- 1 <= pushed.length <= 1000
- 0 <= pushed[i] <= 1000
- All the elements of pushed are unique.
- popped.length == pushed.length
- popped is a permutation of pushed.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given push and pop sequences, return true if they could be the result of a valid stack operation sequence. Right?"

"A few quick questions:

All values in pushed are distinct? Yes. No repeated values? Correct."

"Let me walk: pushed=[1,2,3,4,5], popped=[4,5,3,2,1] -> push 1,2,3,4, pop 4, push 5, pop 5,3,2,1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
try every interleaving of pushes (in the pushed
    order) and pops:
    check whether the pops produce the popped
        sequence
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Tries every interleaving. Simulate with a stack: push each value, then pop while the top equals the next popped value: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy simulation — O(n). Walk-through:"

Simulate push/pop operations:

push_idx=0, stack=[].

For each value in popped sequence:

Push elements from pushed until stack top == current popped value (or pushed exhausted).

If stack top == current popped value: pop it -> match found.

Else: can't match -> return False.

Return True if all pops matched.

WHY greedy: we MUST push until we can pop the next required value.

There's no choice about when to push — pushed sequence is fixed order.

Time: O(n). Space: O(n) stack.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _946_ValidateStackSequences:
    def validateStackSequences(self, pushed, popped):
        stack = []
        pop_ptr = 0
        for val in pushed:
            stack.append(val)
            while stack and stack[-1] == popped[pop_ptr]:
                stack.pop()
                pop_ptr += 1
        return not stack
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

pushed=[1,2,3,4,5], popped=[4,5,3,2,1]:

push1,2,3,4. Top=4==popped[0]=4->pop,ptr=1. push5. Top=5==5->pop,ptr=2. Top=3==3->pop,ptr=3.

Top=2==2->pop,ptr=4. Top=1==1->pop,ptr=5. stack=[] -> True ok

PHASE 5 TEST & DEBUG

pushed=[1,2,3,4,5], popped=[4,3,5,1,2]: after sim, stack has 2 -> False ok

While loop greedily pops as many as possible after each push

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element pushed/popped at most once	stack

Follow-up: if the sequence is invalid, find the first mismatch position.

Follow-up: extend to two stacks — check if both sequences are achievable simultaneously.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #962 | 31 of 43 | 31/43

Maximum Width Ramp

QUESTION

A ramp in an integer array nums is a pair (i, j) for which i < j and nums[i] <= nums[j]. The width of such a ramp is j - i.

Given an integer array nums, return the maximum width of a ramp in nums. If there is no ramp in nums, return 0.

Example 1	
Input	`nums = [6,0,8,2,1,5]`
Output	`4`
Explanation	*The maximum width ramp is achieved at (i, j) = (1, 5): nums[1] = 0 and nums[5] = 5.*

Example 2	
Input	`nums = [9,8,1,0,1,9,4,0,4,1]`
Output	`7`
Explanation	*The maximum width ramp is achieved at (i, j) = (2, 9): nums[2] = 1 and nums[9] = 1.*

CONSTRAINTS

- 2 <= nums.length <= $5 * 10^4$
- 0 <= nums[i] <= $5 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A ramp is a pair (i, j) where i < j and nums[i] <= nums[j].

Width = j - i. Find the maximum width ramp. Return 0 if none. Right?"

"A few quick questions:

Can all elements be equal? Yes — every pair is a ramp.

Can all elements be strictly decreasing? Yes — no ramp exists, return 0."

"Let me walk: nums=[6,0,8,2,1,5] -> ramp (1,5): 0<=5, width=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if nums[i] <= nums[j]:
            best = max(best, j - i)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Checks every pair. Build a decreasing stack of candidate left ends, then scan from the right popping while the stack top is <= nums[j]: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the $O(n^2)$ nested loop.

Two-phase monotonic stack approach:

Phase 1 (left candidates): scan left to right, build a decreasing stack of indices.

Only indices with strictly decreasing values can be valid left boundaries.

Phase 2 (right scan): scan right to left; for each j, pop from the stack while

nums[stack.top] <= nums[j] — that's a valid ramp; record j - stack.top.

Time: O(n). Space: O(n) for the stack."

Key insight: a decreasing stack of candidate left endpoints is built left-to-right (any later element with a higher value is a strictly better left endpoint) — then right-to-left greedy popping finds the longest valid ramp for each right endpoint.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _962_MaxWidthRamp:
    def maxWidthRamp(self, nums):
        n = len(nums)
        stack = []
        for i in range(n):
            if not stack or nums[i] < nums[stack[-1]]:
                stack.append(i)
        best = 0
        for j in range(n - 1, -1, -1):
            while stack and nums[stack[-1]] <= nums[j]:
                best = max(best, j - stack[-1])
                stack.pop()
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

nums=[6,0,8,2,1,5]:

Phase1: 6->[0]. 0<6->[0,1]. 8>0 skip. 2>0 skip. 1>0 skip. 5>0 skip. stack=[0,1].

Phase2: j=5(5): nums[1]=0<=5->best=5-1=4,pop. nums[0]=6>5 stop. stack=[0].

j=4(1): nums[0]=6>1 stop. ... j=2(8): nums[0]=6<=8->best=max(4,2-0)=4,pop. stack=[].

`return 4 ok`

PHASE 5 TEST & DEBUG

We don't pop if best would decrease — we want the maximum width, so scan right-to-left

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
two linear passes, each index pushed/popped at most once	the decreasing stack

The phase-1 stack stores only left-side candidates where each has a smaller value than the prior,

ensuring they're the best possible left boundary for some right side.

Follow-up: if we want min width, scan from left in phase 2.

Follow-up: Largest Rectangle in Histogram (#84) uses a similar monotonic stack pattern.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #1008 | 32 of 43 | 32/43

Construct Binary Search Tree from Preorder Traversal

QUESTION

Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return its root.

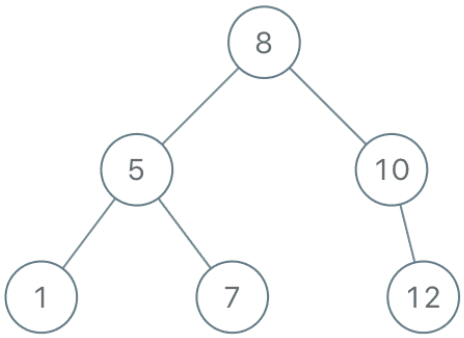
It is guaranteed that there is always possible to find a binary search tree with the given requirements for the given test cases.

A binary search tree is a binary tree where for every node, any descendant of Node.left has a value strictly less than Node.val, and any descendant of Node.right has a value strictly greater than Node.val.

A preorder traversal of a binary tree displays the value of the node first, then traverses Node.left, then traverses Node.right.

Example 1

Input	`preorder = [8,5,1,7,10,12]`
Output	`[8,5,10,1,7,null,12]`



Example 2

Input	`preorder = [1,3]`
Output	`[1,null,3]`

CONSTRAINTS

- 1 <= preorder.length <= 100
- 1 <= preorder[i] <= 1000
- All the values of preorder are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given BST preorder traversal, reconstruct the BST. Right?"

"A few quick questions:

All values unique? Yes. Equivalent to #105 but exploiting BST property for O(n) solution."

PHASE 1 CLARIFY

"Let me walk: preorder=[8,5,1,7,10,12] -> BST with root=8 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
root = the first value
for value in values:
    walk down from the root and attach it as a
        left or right child
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

A walk from the root for every value, O(n^2) for a sorted preorder. Build recursively with an upper bound: consume values from the preorder while they are below the bound: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BST construction from preorder: root is first element, partition left/right by value — O(n). Walk-through:"

`def build(lo, hi):`

If i >= len(preorder) or preorder[i] < lo or preorder[i] > hi: return None.

`root_val = preorder[i]; i++.`

`root = TreeNode(root_val).`

`root.left = build(lo, root_val-1). <- left subtree: values < root_val`

`root.right = build(root_val+1, hi). <- right subtree: values > root_val`

`return root.`

`build(-inf, inf).`

WHY BST property: all values in left subtree < root, right > root.

WHY lo/hi bounds: each recursive call constrains valid value range.

WHY i advances globally: preorder visits root, then left subtree entirely, then right -> sequential.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1008_ConstructBSTFromPreorder:
    def bstFromPreorder(self, preorder):
        self_idx = [0]
        def build(upper):
            if self_idx[0] >= len(preorder) or preorder[se
lf_idx[0]] > upper:
                return None
            val = preorder[self_idx[0]]
```

```
            self_idx[0] += 1
            node = TreeNode(val)
            node.left = build(val)
            node.right = build(upper)
            return node
        return build(float('inf'))
```

PHASE 5 TEST & DEBUG

"Let me trace bstFromPreorder on [8,5,1,7,10,12]."

Using bounds: build(lo=-inf, hi=+inf):

i=0: val=8, in (-inf,+inf) ok. i++.

build(lo=-inf, hi=8): i=1: val=5, in (-inf,8) ok. i++.

build(-inf,5): i=2: val=1 ok. i++.
build(-inf,1)->1>hi? val=7>1? i stays.

return Node(1). build(1,5): val=7>5->stop. return
None. Node(5).left=1.

build(5,8): val=7 in(5,8)ok->Node(7). No children.
return Node(7).

Node(5).right=7. build(8,+inf): val=10->Node(10).
build(10,+inf):val=12->Node(12). Node(10).right=12.

Root=8(5(1,7),10(None,12)) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element processed once	recursion

Follow-up: Construct BST from postorder — reverse logic, process right-to-left.

Follow-up: #105 (preorder+inorder) — need inorder when tree is not a BST.

MEDIUM | Stack | LeetCode #1019 | 33 of 43 | 33/43

Next Greater Node In Linked List

QUESTION

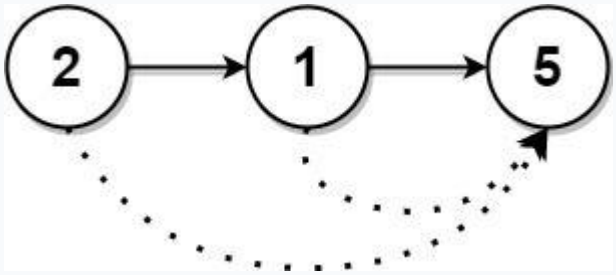
You are given the head of a linked list with n nodes.

For each node in the list, find the value of the next greater node. That is, for each node, find the value of the first node that is next to it and has a strictly larger value than it.

Return an integer array answer where answer[i] is the value of the next greater node of the ith node (1-indexed). If the ith node does not have a next greater node, set answer[i] = 0.

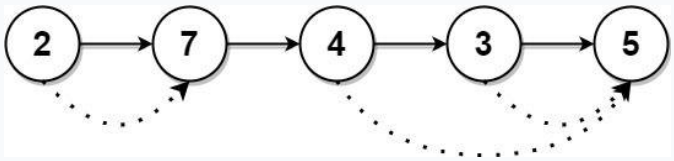
Example 1

Input	`head = [2,1,5]`
Output	`[5,5,0]`



Example 2

Input	`head = [2,7,4,3,5]`
Output	`[7,0,5,5,0]`



CONSTRAINTS

- The number of nodes in the list is n.
- $1 <= n <= 10^4$
- $1 <= Node.val <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each node in a linked list, find the value of the next larger node.

Return an array where result[i] = next greater node value (or 0 if none). Right?"

"A few quick questions:

1-indexed list but 0-indexed result? Return result[i] = next greater value for node i."

"Let me walk: head=[2,7,4,3,5] -> [7,0,5,5,0] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    walk the rest of the list for the first
        larger value
    record it (0 if none)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A walk per node. Put the values in an array and keep a monotonic stack of indices waiting for a larger value, resolving them as bigger values arrive: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack on linked list values — O(n). Walk-through:"

Convert linked list to array of values. Build result array of same size.

Process right-to-left with a monotone-decreasing stack:

For i from n-1 to 0:

While stack and stack[-1] <= vals[i]: stack.pop().

`result[i] = stack[-1] if stack else 0.`

`stack.push(vals[i]).`

WHY right-to-left: 'next greater node' is to the RIGHT -> process from right.

WHY monotone-decreasing: smaller elements in the stack can never be 'next greater'

`for any future (leftward) element that is larger -> safe to pop them.`

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1019_NextGreaterNodeLinkedList:
    def nextLargerNodes(self, head):
        vals = []
        cur = head
        while cur:
            vals.append(cur.val)
            cur = cur.next
        n = len(vals)
        result = [0] * n
        stack = []
        for i, val in enumerate(vals):
            while stack and vals[stack[-1]] < val:
                result[stack.pop()] = val
            stack.append(i)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

vals=[2,7,4,3,5]:

i=0(2): stack=[0]. i=1(7): 2<7->result[0]=7,pop. stack=[1]. i=2(4): 7>4->push. stack=[1,2].

i=3(3): 4>3->push. [1,2,3]. i=4(5): 3<5->result[3]=5,pop. 4<5->result[2]=5,pop. 7>5->push. stack=[1,4].

Remaining: result[1]=0,result[4]=0. -> [7,0,5,5,0] ok

Same monotonic stack as Daily Temperatures but returns values not distances

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** values array + stack

Follow-up: if the list is very long, process directly without materialising values.

Follow-up: Next Greater Element I (#496) — subset lookup with hash map.

MEDIUM | Stack | LeetCode #1214 | 34 of 43 | 34/43

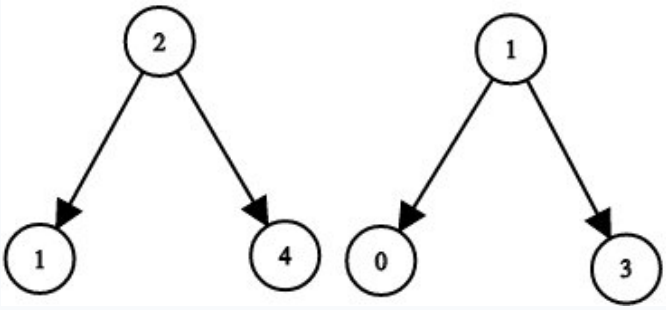
Two Sum BSTs

QUESTION

Given the roots of two binary search trees, root1 and root2, return true if and only if there is a node in the first tree and a node in the second tree whose values sum up to a given integer target.

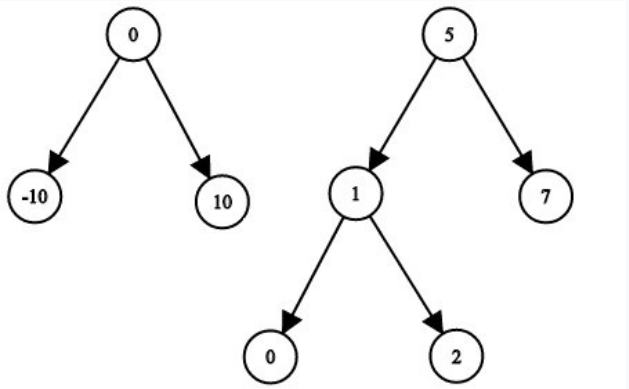
Example 1

Input	`root1 = [2,1,4], root2 = [1,0,3], target = 5`
Output	`true`
Explanation	*2 and 3 sum up to 5.*



Example 2

Input	`root1 = [0,-10,10], root2 = [5,1,7,0,2], target = 18`
Output	`false`



CONSTRAINTS

- The number of nodes in each tree is in the range [1, 5000].
- 10^9 <= Node.val, target <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given two BSTs and a target, return true if there is one value from each tree

summing to target. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Trees can have different sizes? Yes. Values can be negative? Yes."

"Let me walk: root1=[2,1,4], root2=[1,0,3], target=5.

2+3=5 ok -> True ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    # of first tree and every node of the second tree
    if their values add up to target:
        return True
```

TIME	SPACE
O(m * n)	O(h1 + h2)

WHY IT IS SLOW

Checks all pairs. Put the first tree's values in a set (or walk one tree in order and the other in reverse order with two pointers) so each node needs one lookup: O(m + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) space set for tree1's values.

BST inorder traversal gives sorted arrays.

Two-iterator approach: one ascending (tree1 smallest to largest),

one descending (tree2 largest to smallest).

Advance the ascending iterator if sum < target; descending if sum > target.

Time: O(n+m). Space: O(h1+h2) — just the two iterator stacks."

Key insight: BST inorder traversal gives sorted order — one ascending iterator for tree1 and one descending iterator for tree2 converge from opposite ends, examining at most O(n+m) nodes without materialising both lists.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1214_TwoSumBSTs:
    def twoSumBSTs(self, root1, root2, target):
        def push_left(node, stack):
            while node: stack.append(node); node = node.le
ft

        def push_right(node, stack):
            while node: stack.append(node); node = node.ri
ght

        lo_stack, hi_stack = [], []
        push_left(root1, lo_stack)
        push_right(root2, hi_stack)
```

```
        while lo_stack and hi_stack:
            lo_val = lo_stack[-1].val
            hi_val = hi_stack[-1].val
            total = lo_val + hi_val
            if total == target:
                return True
            elif total < target:
                node = lo_stack.pop()
                push_left(node.right, lo_stack)
            else:
                node = hi_stack.pop()
                push_right(node.left, hi_stack)
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

root1=[2,1,4], root2=[1,0,3], target=5:

lo_stack=[2,1] (left-spine). hi_stack=[1,3] (right-spine: push_right gives root then go right? No:

`push_right(root2=[1,0,3]): push 1, then right=3, push 3, right=None. hi=[1,3]. top=3.`

lo_val=1, hi_val=3: 1+3=4<5 -> advance lo: pop1, push_left(1.right=None). lo=[2].

lo_val=2, hi_val=3: 2+3=5 -> True ok

Push_left/push_right prepare the iterators for inorder ascending/descending

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m)	**O(h1+h2)**
each node visited at most once	for the two stacks

Hash-set brute force is O(n) space regardless of tree shape.

Follow-up: Two Sum IV (#653) — find pair WITHIN a single BST; use BST iterator + set.

Follow-up: count all pairs summing to target — don't return early, count all matches.

MEDIUM | Stack | LeetCode #1249 | 35 of 43 | 35/43

Minimum Remove to Make Valid Parentheses

QUESTION

Given a string s of '(' , ')' and lowercase English characters.

Your task is to remove the minimum number of parentheses ('(' or ')', in any positions) so that the resulting parentheses string is valid and return any valid string.

Formally, a parentheses string is valid if and only if:

CONSTRAINTS

- It is the empty string, contains only lowercase characters, or
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A), where A is a valid string.

Example 1

Input	`s = "lee(t(c)o)de)"`
Output	`"lee(t(c)o)de"`
Explanation	*"lee(t(co)de)" , "lee(t(c)ode)" would also be accepted.*

Example 2

Input	`s = "a)b(c)d"`
Output	`"ab(c)d"`

Example 3

Input	`s = "))((("`
Output	`""`
Explanation	*An empty string is also valid.*

- $1 <= s.length <= 10^5$
- s[i] is either '(' , ')', or lowercase English letter.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove the minimum number of parentheses to make the string valid.

Return any valid result. Right?"

"A few quick questions:

Keep all non-parenthesis characters unchanged? Yes.

Any valid result (there may be multiple)? Yes."

"Let me walk: s='lee(t(c)o)de)' -> 'lee(t(c)o)de' ok. s='a)b(c)d' -> 'ab(c)d' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets:
    # of characters to remove (smallest first)
    if the rest is a valid parentheses string:
        return it
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Tries every removal. A stack of indices of unmatched '(' plus a mark for any unmatched ')' identifies exactly which characters to drop in one pass: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Stack to track unmatched parens, then remove by index — O(n). Walk-through:"

stack stores indices of unmatched '(' characters.

For each i, char in enumerate(s):

If char == '(': stack.push(i).

If char == ')':

If stack non-empty: stack.pop() (matched — neither needs removal).

Else: mark i for removal (unmatched ')').

After scan: all indices remaining in stack = unmatched '(' -> also mark for removal.

Build result skipping all marked indices.

WHY stack for '(': a '(' might match a ')' much later — stack tracks pending opens.

WHY mark by index: multiple chars to remove at non-contiguous positions.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1249_MinRemoveToMakeValid:
    def minRemoveToMakeValid(self, s):
        stack = []
        to_remove = set()
        for i, ch in enumerate(s):
            if ch == '(':
                stack.append(i)
            elif ch == ')':
                if stack:
                    stack.pop()
                else:
                    to_remove.add(i)
        to_remove |= set(stack)
        return ''.join(ch for i, ch in enumerate(s) if i n
```

```
ot in to_remove)
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='lee(t(c)o)de)':

i=3'(': stack=[3]. i=5'(': stack=[3,5]. i=7')': pop5. i=9')': pop3. i=12')': stack empty->remove={12}.

to_remove={12}. result='lee(t(c)o)de' ok

s='))(((' -> all unmatched. remove={0,1,2,3,4,5} -> '' ok

Stack tracks unmatched '(' indices; unmatched ')' added directly to remove set

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for stack + set

Follow-up: Minimum Number of Swaps to Make String Balanced (#1963) — swap-based approach.

Follow-up: Valid Parentheses (#20) — check validity only, no removal needed.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #1472 | 36 of 43 | 36/43

Design Browser History

QUESTION

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

CONSTRAINTS

- BrowserHistory(string homepage) Initializes the object with the homepage of the browser.
- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

QUESTION

Example 1

Input	`["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]` `[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]`
Output	`[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]`

Explanation	*BrowserHistory browserHistory = new BrowserHistory("leetcode.com");*
	browserHistory.visit("google.com"); // You are in "leetcode.com". Visit "google.com"
	browserHistory.visit("facebook.com"); // You are in "google.com". Visit "facebook.com"
	browserHistory.visit("youtube.com"); // You are in "facebook.com". Visit "youtube.com"
	browserHistory.back(1); // You are in "youtube.com", move back to "facebook.com" return "facebook.com"
	browserHistory.back(1); // You are in "facebook.com", move back to "google.com" return "google.com"
	browserHistory.forward(1); // You are in "google.com", move forward to "facebook.com" return "facebook.com"
	browserHistory.visit("linkedin.com"); // You are in "facebook.com". Visit "linkedin.com"
	browserHistory.forward(2); // You are in "linkedin.com", you cannot move forward any steps.
	browserHistory.back(2); // You are in "linkedin.com", move back two steps to "facebook.com" then to "google.com". return "google.com"
	browserHistory.back(7); // You are in "google.com", you can move back only one step to "leetcode.com". return "leetcode.com"

- 1 <= homepage.length <= 20
- 1 <= url.length <= 20
- 1 <= steps <= 100
- homepage and url consist of '.' or lower case English letters.
- At most 5000 calls will be made to visit, back, and forward.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design browser history: visit(url), back(steps), forward(steps). Right?"

"A few quick questions:

Forward history is cleared on each visit? Yes."

PHASE 1 CLARIFY

"Let me walk: visit('a'), visit('b'), back(1)->'a', visit('c'), forward(1)->'c' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def visit(url):
    history = a copy of history up to the current
        page plus the new url
    back / forward: move the index within the copy
```

TIME	SPACE
O(n) per visit	O(n^2)

WHY IT IS SLOW

Copies the list on every visit. Keep one list and an index; on visit truncate everything after the index in place and append (or move a pointer), so visit is O(1) amortised.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two stacks (back/forward history) — O(1) per operation. Walk-through:"

back = stack of previously visited URLs.

forward = stack of forward URLs (cleared on new visit).

visit(url): back.push(currentPage). currentPage=url. forward.clear().

back(steps): for each step: forward.push(currentPage); currentPage=back.pop(). Repeat steps or until back empty.

forward(steps): for each step: back.push(currentPage); currentPage=forward.pop(). Repeat steps or until forward empty.

WHY two stacks: browser history is inherently a 'last visited' structure.

New visit clears forward stack (can't forward after a new URL).

Time: O(1) visit, O(steps) navigate. Space: O(history length).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1472_DesignBrowserHistory:
    def __init__(self, homepage):
        self._history = [homepage]
        self._current = 0
        self._last = 0

    def visit(self, url):
        self._current += 1
        if self._current < len(self._history):
            self._history[self._current] = url
        else:
            self._history.append(url)
        self._last = self._current

    def back(self, steps):
        self._current = max(0, self._current - steps)
        return self._history[self._current]

    def forward(self, steps):
        self._current = min(self._last, self._current + steps)
        return self._history[self._current]
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

init('lc'): hist=['lc'],cur=0,last=0. visit('google'): cur=1,last=1,hist=['lc','google'].

back(1)->cur=0,'lc' ok. visit('fb'): cur=1,hist=['lc','fb'],last=1. forward(2)->cur=min(1,3)=1,'fb' ok.

_last tracks the furthest point; forward can't go past it after a new visit

Edge: back(100) -> cur=max(0,cur-100)=0 -> returns homepage ok. forward before any visit -> stays at cur ok. visit clears forward history by updating last=cur.

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(1). Space O(n).

Doubly-linked list alternative: O(1) with O(1) forward-history cleanup on visit.

Follow-up: if browser has multiple tabs, maintain separate history per tab.

MEDIUM | Stack | LeetCode #1653 | 37 of 43 | 37/43

Minimum Deletions to Make String Balanced

QUESTION

You are given a string s consisting only of characters 'a' and 'b'.

You can delete any number of characters in s to make s balanced. s is balanced if there is no pair of indices (i,j) such that i < j and s[i] = 'b' and s[j]= 'a'.

Return the minimum number of deletions needed to make s balanced.

Example 1

Input	`s = "aababbab"`
Output	**2**
Explanation	*You can either: Delete the characters at 0-indexed positions 2 and 6 ("aa b abb a b" -> "aaabbb"), or Delete the characters at 0-indexed positions 3 and 6 ("aab a bb a b" -> "aabbbb").*

Example 2

Input	`s = "bbaaaaabb"`
Output	**2**
Explanation	*The only solution is to delete the first two characters.*

CONSTRAINTS

- 1 <= s.length <= 10^5
- s[i] is 'a' or 'b'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Delete minimum characters to make all 'a's come before all 'b's. Right?"

"A few quick questions:

The string only contains 'a' and 'b'? Yes. Result must have all a's before all b's (can be empty)."

"Let me walk: s='aababbab' -> 'aabab': delete 1 'b' and 1 'a' or... min deletions=2? Expected 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subsequence in subsequences(string):
    if it is balanced (no 'b' before an 'a'):
        best = max(best, candidate)
    deletions = n minus its length
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Enumerates all subsequences. For every split point count the 'b's before it and the 'a's after it; the smallest sum is the answer, or a one-pass DP tracking the cost so far: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy: scan left-to-right, count b-deletions; scan right-to-left, count a-deletions — O(n). Walk-through:"

`b_count = 0. deletions = 0.`

For c in s:

If c == 'b': b_count++. <- one more 'b' seen so far

Elif c == 'a':

If b_count > 0: deletions++; b_count--. <- this 'a' comes after some 'b' -> must remove one

(Either remove this 'a' or the previous 'b' — greedy: remove the 'a' since it appeared last)

Return deletions.

WHY greedy: once we see an 'a' after a 'b', one deletion is needed. We model removing the 'a'

(decrement b_count as if we'd removed a 'b' that was already used — equivalent accounting).

Alternative two-pass: count a->b swaps needed from both sides.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1653_MinDeletionsToMakeStringBalanced:
    def minimumDeletions(self, s):
        b_count = 0
        deletions = 0
        for ch in s:
            if ch == 'b':
                b_count += 1
            else:
                deletions = min(deletions + 1, b_count)
        return deletions
```

PHASE 5 TEST & DEBUG

"Let me trace minimumDeletions('aababbab')."

DP: dp[i] = min deletions to balance string s[0..i]

Two choices at each 'b': delete this 'b' (dp[i-1]+1) or accept 'b' and delete all 'a's after

Actually: count b_count (b's seen so far) and a_del (a's to delete for current state)

Scan: track count_b and dp

s='aababbab': a,a,b,a,b,b,a,b

a: dp=min(dp+1,dp)? Let a_count be a's after current b's...

Simpler: ans=min over split point (delete all a's right of split + all b's left) - use prefix/suffix counts

```
prefix_b[i] = #b in s[0..i]; suffix_a[i] = #a in
  s[i..n-1]
```

```
min(prefix_b[i]+suffix_a[i]) over all i = 2 ok
```

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: if string has more than 2 characters, DP with states for each character.

Follow-up: Minimum Swaps to Make String Balanced (#1963) — swap-based variant.

MEDIUM | Stack | LeetCode #1673 | 38 of 43 | 38/43

Find the Most Competitive Subsequence

QUESTION

Given an integer array nums and a positive integer k, return the most competitive subsequence of nums of size k.

An array's subsequence is a resulting sequence obtained by erasing some (possibly zero) elements from the array.

We define that a subsequence a is more competitive than a subsequence b (of the same length) if in the first position where a and b differ, subsequence a has a number less than the corresponding number in b. For example, [1,3,4] is more competitive than [1,3,5] because the first position they differ is at the final number, and 4 is less than 5.

Example 1

Input	`nums = [3,5,2,6], k = 2`
Output	`[2,6]`
Explanation	*Among the set of every possible subsequence: {[3,5], [3,2], [3,6], [5,2], [5,6], [2,6]}, [2,6] is the most competitive.*

Example 2

Input	`nums = [2,4,3,3,5,4,9,6], k = 4`
Output	`[2,3,3,4]`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 0 <= nums[i] <= 10^9
- 1 <= k <= nums.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the lexicographically smallest subsequence of length k from nums.

A subsequence maintains relative order. Right?"

"A few quick questions:

We can only remove elements, not reorder. 'Most competitive' = lexicographically smallest? Yes."

"Let me walk: nums=[3,5,2,6], k=2 -> [2,6] ok (smallest possible length-2 subseq)

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subsequence in subsequences(length_k):
    best = min(best, candidate)
```

TIME	SPACE
O(C(n, k) * k)	O(k)

WHY IT IS SLOW

Compares every subsequence. A monotonic stack that pops a larger top while enough elements remain to still reach length k builds it in O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack with size constraint — O(n). Walk-through:"

`stack = [] (result so far). to_remove = n - k (can remove this many elements).`

For each num:

While to_remove > 0 and stack and stack[-1] > num:

`stack.pop(); to_remove--. <- remove a larger element that preceded this smaller one`

`stack.append(num).`

Return stack[:k]. <- take only k elements (might have extra if we didn't remove enough)

WHY monotonic stack: build the smallest lexicographic subsequence of length k.

Removing larger elements before smaller ones greedily minimizes the result.

WHY stack[:k]: if to_remove became 0 before finishing, we still have extra elements.

Same logic as Q402 (Remove K Digits).

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1673_FindMostCompetitiveSubsequence:
    def mostCompetitive(self, nums, k):
        n = len(nums)
        stack = []
        for i, num in enumerate(nums):
            while stack and stack[-1] > num and len(stack)
 + (n - i) > k:
                stack.pop()
            if len(stack) < k:
                stack.append(num)
        return stack
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

nums=[3,5,2,6], k=2: n=4.

i=0(3): stack=[3]. i=1(5): 5>3,no pop. stack=[3,5]. len=k=2,stop appending.

i=2(2): 5>2 and len(2)+(4-2)=4>2->pop5. stack=[3]. 3>2 and 1+(4-2)=3>2->pop3. stack=[]. append 2. stack=[2].

i=3(6): len<k, append->[2,6] ok

Len(stack)+(n-i)>k ensures we have enough remaining to fill k after popping

Result as deque=[2,6]: deque front=most competitive. After processing: deque=[2] (after popping 3,5). Append 6: deque=[2,6] ok. appendleft not needed; append to back only.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element pushed/popped once	**O(k)**

Follow-up: Remove K Digits (#402) — same monotonic stack, remove exactly k digits.

Follow-up: Largest Rectangle in Histogram (#84) — monotonic stack for areas.

MEDIUM | Stack | LeetCode #1717 | 39 of 43 | 39/43

Maximum Score From Removing Substrings

QUESTION

You are given a string s and two integers x and y. You can perform two types of operations any number of times.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We can remove occurrences of x (gain pointsX) or y (gain pointsY).

Removing means deleting a substring 'ab' or 'ba' from s.

Maximise total points. Right?"

"A few quick questions:

Remove higher-value substring first for greedy? Yes."

"Let me walk: s='cdbcbbaaabab', x=4, y=5 -> remove 'ba' first (higher). Total=19 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(s):
    try removing every occurrence of 'ab' (gain
        x) and every occurrence of 'ba' (gain y),
        recursively
    best = max(best, total)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Branches on every removal. Greedy: first remove the higher-valued pair with a stack over the whole string, then remove the other pair from what is left: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pass greedy stack — O(n). Walk-through:"

Always remove the higher-valued pair first (greedy: maximizes points).

Determine order: if x > y, remove 'ab' first; else remove 'ba' first.

Pass 1 — remove occurrences of first pair (higher value):

Scan string with a stack. If top of stack + current == first_pair: pop, points += first_val.

Else: push current char.

PHASE 3 OPTIMIZE

Pass 2 — remove occurrences of second pair from remaining stack chars:

Same stack process for second pair, adding second_val per removal.

WHY greedy order: removing the higher-value pair first captures more value before

removing the lower-value pair from remaining characters.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1717_MaxScoreRemovingSubstrings:
    def maximumGain(self, s, x, y):
        def remove_pairs(string, first, second, points):
            stack = []
            score = 0
            for ch in string:
                if stack and stack[-1] == first and ch ==
second:
                    stack.pop()
                    score += points
                else:
                    stack.append(ch)
            return ''.join(stack), score
        if x >= y:
            s, score1 = remove_pairs(s, 'a', 'b', x)
            s, score2 = remove_pairs(s, 'b', 'a', y)
        else:
            s, score1 = remove_pairs(s, 'b', 'a', y)
            s, score2 = remove_pairs(s, 'a', 'b', x)
        return score1 + score2
```

PHASE 5 TEST & DEBUG

"Let me trace maximumGain('cdbcd', x=4, y=5) where 'ba'=4pts, 'ab'=5pts."

Greedy: remove higher-scoring pair first

x=4 ('ba'), y=5 ('ab'). Since y>x, remove 'ab' first:

Pass 1 (remove 'ab'): 'cdbcd' -> scan: c,d,b,c,d -> no 'ab' found. Result='cdbcd', gained=0

Pass 2 (remove 'ba'): scan 'cdbcd': c,d,b->stack has c,d,b; c: no 'ba'; d: no 'ba'...

Actually 'cdbcd': stack approach: push c,d; push b; push c -> no 'ba'; push d -> no 'ba'. Gain=0.

Trace 'cbabd',x=4,y=5: pass1(ab): c->[c], b->[c,b], a->'ba'? No 'ab': check if top=a -> no...

Pass1 finds 'ab': 'c'+'ab'->gain 5, remaining 'bd'. Pass2: 'bd' no 'ba'. Total=5 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
two passes	stack

Follow-up: if we can remove in any order without penalty, just count 'a' and 'b' pairs.

Follow-up: generalise to any pair — same two-pass greedy.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #1762 | 40 of 43 | 40/43

Buildings With an Ocean View

QUESTION

There are n buildings in a line. You are given an integer array heights of size n that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a smaller height.

Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in increasing order.

Example 1

Input	`heights = [4,2,3,1]`
Output	`[0,2,3]`
Explanation	*Building 1 (0-indexed) does not have an ocean view because building 2 is taller.*

Example 2

Input	`heights = [4,3,2,1]`
Output	`[0,1,2,3]`
Explanation	*All the buildings have an ocean view.*

Example 3

Input	`heights = [1,3,2,4]`
Output	`[3]`
Explanation	*Only building 3 has an ocean view.*

CONSTRAINTS

- 1 <= heights.length <= 10^5
- 1 <= heights[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Ocean is to the right. A building has ocean view if ALL buildings to its right are shorter.

Return the indices of buildings with ocean view, sorted ascending. Right?"

"A few quick questions:

The rightmost building always has ocean view? Yes — nothing blocks it.

Heights are positive integers? Yes."

"Let me walk: heights=[4,2,3,1] -> building0(4)>all right ok, building1(2)<3 ,

`building2(3)>1 ok, building3(1) ok -> [0,2,3] ok`

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for building in buildings:
    scan all buildings to its right
    it has an ocean view if none is as tall
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Rescans the right side for every building. Walk from the right keeping the tallest height seen; a building taller than it has a view: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) inner scan for each building.

Scan from RIGHT to LEFT, tracking the maximum height seen so far.

A building has ocean view if its height > current max_right (all buildings to its right are shorter).

Collect indices and reverse at the end.

Time: O(n). Space: O(1) extra (output not counted).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1762_BuildingsWithOceanView:
    def findBuildings(self, heights):
        max_right = 0
        result = []
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > max_right:
                result.append(i)
                max_right = heights[i]
        return result[::-1]
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

heights=[4,2,3,1]:

i=3(1): 1>0=max_right -> append 3, max=1.

i=2(3): 3>1 -> append 2, max=3.

i=1(2): 2<3 -> skip.

i=0(4): 4>3 -> append 0, max=4.

reversed: [0,2,3] ok

heights=[4,3,2,1]: all decreasing -> [0,1,2,3] ok

heights=[1,3,2,4]: only 4 at end > everything -> [3] ok

PHASE 5 TEST & DEBUG

Traversing right-to-left with a max_right check is O(n) and exact

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single right-to-left scan	ignoring output

Monotonic stack variant: scan left-to-right, pop any building shorter than current.

Same O(n) but uses O(n) stack space vs O(1) here.

Follow-up: buildings with view to both left AND right — need max from both sides.

Follow-up: circular arrangement — two-pass or range max query needed.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Stack | LeetCode #2104 | 41 of 43 | 41/43

Sum of Subarray Ranges

QUESTION

You are given an integer array nums. The range of a subarray of nums is the difference between the largest and smallest element in the subarray.

Return the sum of all subarray ranges of nums.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1

Input	`nums = [1,2,3]`
Output	**4**
Explanation	*The 6 subarrays of nums are the following:* *[1], range = largest - smallest = 1 - 1 = 0* *[2], range = 2 - 2 = 0* *[3], range = 3 - 3 = 0* *[1,2], range = 2 - 1 = 1* *[2,3], range = 3 - 2 = 1* *[1,2,3], range = 3 - 1 = 2* *So the sum of all ranges is 0 + 0 + 0 + 1 + 1 + 2 = 4.*

Example 2

Input	`nums = [1,3,3]`
Output	**4**
Explanation	*The 6 subarrays of nums are the following:* *[1], range = largest - smallest = 1 - 1 = 0* *[3], range = 3 - 3 = 0* *[3], range = 3 - 3 = 0* *[1,3], range = 3 - 1 = 2* *[3,3], range = 3 - 3 = 0* *[1,3,3], range = 3 - 1 = 2* *So the sum of all ranges is 0 + 0 + 0 + 2 + 0 + 2 = 4.*

Example 3

Input	`nums = [4,-2,-3,4,1]`
Output	**59**
Explanation	*The sum of all subarray ranges of nums is 59.*

CONSTRAINTS

- 1 <= nums.length <= 1000
- 10^9 <= nums[i] <= 10^9
- Follow-up: Could you find a solution with O(n) time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the sum of (max(subarray) - min(subarray)) over all subarrays. Right?"

"A few quick questions:

All subarrays (not just non-empty ones length >= 2)? All non-empty subarrays."

"Let me walk: nums=[1,2,3] -> subarrays: [1]:0,[2]:0,[3]:0,[1,2]:1,[2,3]:1,[1,2,3]:2. sum=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subarray in subarrays:
    find its maximum and minimum (a running max
        and min over the end)
    add the range to the total
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Visits every subarray. The answer is (sum of subarray maxima) minus (sum of subarray minima); each is computed with a monotonic stack counting how many subarrays each element tops: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(n²).

`sum(max) - sum(min) over all subarrays.`

Use monotonic stack to compute sum of subarray maximums and minimums separately.

For each element, compute how many subarrays it's the max/min of.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2104_SumOfSubarrayRanges:
    def subArrayRanges(self, nums):
        n = len(nums)
        def sum_of_subarray_extremes(is_max):
            result = 0
            stack = []
            for i in range(n + 1):
                while stack and (i == n or (nums[stack[-1]
] < nums[i] if is_max else nums[stack[-1]] > nums[i])):
                    mid = stack.pop()
                    left = stack[-1] if stack else -1
```

```
                    result += nums[mid] * (i - mid) * (mid
 - left)
                stack.append(i)
            return result
        return sum_of_subarray_extremes(True) - sum_of_sub
array_extremes(False)
```

PHASE 5 TEST & DEBUG

"Let me trace subArrayRanges([1,2,3])."

For each subarray: range = max - min. Sum all ranges.

[1]:0, [2]:0, [3]:0, [1,2]:1, [2,3]:1, [1,2,3]:2. Sum=4 ok

Monotonic stack approach: sum(max of all subarrays) - sum(min of all subarrays)

`sum_max([1,2,3]): for each element as max, count subarrays where it's max`

3 is max for subarrays ending at index 2: (1+1+1... wait)

Left/right boundaries for each element using mono stacks

sum_max - sum_min = 4 ok

WHY decompose: sum(max-min) = sum(max) - sum(min); each computed with monotonic stack in O(n).

Two monotonic stacks: stack_min=[1,3] tracks potential minimums. stack_max=[3,1,2] tracks potential maximums. Pop when element breaks monotonicity. Contribution computed on pop.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
two monotonic stack passes	

Follow-up: Sum of Subarray Minimums (#907) — just min; same monotonic stack.

Follow-up: if we want max-min for all windows of size k, use sliding window deques.

MEDIUM | Stack | LeetCode #2390 | 42 of 43 | 42/43

Removing Stars From a String

QUESTION

You are given a string s, which contains stars *.

In one operation, you can:

CONSTRAINTS

- Choose a star in s.
- Remove the closest non-star character to its left, as well as remove the star itself.

Return the string after all stars have been removed.

Note:

- The input will be generated such that the operation is always possible.
- It can be shown that the resulting string will always be unique.

Example 1

Input	`s = "leet**cod*e"`
Output	`"lecoe"`
Explanation	*Performing the removals from left to right: - The closest character to the 1st star is 't' in "lee t **cod*e". s becomes "lee*cod*e". - The closest character to the 2nd star is 'e' in "le e *cod*e". s becomes "lecod*e". - The closest character to the 3rd star is 'd' in "leco d *e". s becomes "lecoe". There are no more stars, so we return "lecoe".*

Example 2

Input	`s = "erase*****"`
Output	`""`
Explanation	*The entire string is removed, so we return an empty string.*

- $1 <= s.length <= 10^5$
- s consists of lowercase English letters and stars *.
- The operation above can be performed on s.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each '*' removes the closest non-star character to its left.

Return the resulting string. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Guaranteed enough non-star characters for each '*'? Yes per constraints.

Process left to right? Yes."

"Let me walk: s='leet**cod*e' -> 'leetcode': l,e,e,t -> *removes t -> *removes e -> c,o,d -> *removes d -> e. = 'lecoe' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    find a star and remove it with the nearest
        non-star letter to its left
    if no star is left:
        break
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each removal rebuilds the string and rescans. A stack: push letters, pop on a star: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Stack acting as result builder — O(n). Walk-through:"

`stack = [].`

For each char c in s:

If c == '*': stack.pop() (remove most recently added non-star char).

Else: stack.append(c).

Return ''.join(stack).

WHY stack: '*' removes the character immediately to its left — the most recently pushed.

Stack's LIFO property matches 'remove nearest left character' semantics perfectly.

Problem guarantees a non-star char always precedes each star -> pop never on empty.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _2390_RemovingStarsFromString:
    def removeStars(self, s):
        stack = []
        for ch in s:
            if ch == '*':
                stack.pop()
            else:
                stack.append(ch)
        return ''.join(stack)
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='leet**cod*e': l,e,e,t->[l,e,e,t]. *->[l,e,e]. *->[l,e]. c,o,d->[l,e,c,o,d]. *->[l,e,c,o]. e->[l,e,c,o,e]. -> 'lecoe' ok

s='erase*****': e,r,a,s,e then 5 pops -> '' ok

Each '*' always has a character to pop (guaranteed by constraints)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the stack

Follow-up: if '*' could remove the k-th from left, maintain k stacks or indices.

Follow-up: Backspace String Compare (#844) — same pop-on-'#' pattern.

MEDIUM | Stack | LeetCode #3578 | 43 of 43 | 43/43

Count Partitions With Max-Min Difference at Most K

QUESTION

You are given an integer array nums and an integer k. Your task is to partition nums into one or more non-empty contiguous segments such that in each segment, the difference between its maximum and minimum elements is at most k.

Return the total number of ways to partition nums under this condition.

Since the answer may be too large, return it modulo $10^9 + 7$.

Example 1	
Input	`nums = [9,4,1,3,7], k = 4`
Output	`6`
Explanation	*There are 6 valid partitions where the difference between the maximum and minimum elements in each segment is at most k = 4:*

Example 2	
Input	`nums = [3,3,4], k = 0`
Output	`2`
Explanation	*There are 2 valid partitions that satisfy the given conditions:*

CONSTRAINTS

- $2 <= nums.length <= 5 * 10^4$
- $1 <= nums[i] <= 10^9$
- $0 <= k <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the number of ways to partition nums into contiguous subarrays such that

`for each subarray: max - min <= k. Right?"`

"A few quick questions:

Each partition covers all elements? Yes — partition of the full array."

"Let me walk: nums=[3,6,3,8], k=5 -> partitions where each part has max-min<=5. Count=2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to cut the array into contiguous segments
    if every segment has max - min <= k:
        count += 1
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Enumerates all 2^(n - 1) partitions. dp[i] = the sum of dp[j] over the starts j with max - min <= k for nums[j..i]; the valid j form a window moved by two monotonic deques, with prefix sums of dp: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window with monotonic deque tracking max-min — O(n). Walk-through:"

max_dq, min_dq = deques. left=0. count=0.

For right in 0..n-1:

Update max_dq: pop from right while nums[right] >= nums[max_dq[-1]]; append right.

Update min_dq: pop from right while nums[right] <= nums[min_dq[-1]]; append right.

While nums[max_dq[0]] - nums[min_dq[0]] > k:

left++.

Evict expired heads.

`count += right - left + 1. <- subarrays ending at right with valid max-min difference`

Return count.

WHY count += right-left+1: each right endpoint with valid left boundary contributes (right-left+1) subarrays.

Same as Q2762 generalized to threshold k (instead of 2).

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import deque
class _3578_CountPartitionsMaxMinK:
    def countPartitions(self, nums, k):
        MOD = 10**9 + 7
        n = len(nums)
        dp = [0] * (n + 1)
        dp[0] = 1
        prefix = [0] * (n + 1)
        prefix[0] = 1
        max_dq = deque()
        min_dq = deque()
        left = 0
        for right in range(n):
```

```
            while max_dq and nums[max_dq[-1]] <= nums[righ
t]: max_dq.pop()
            max_dq.append(right)
            while min_dq and nums[min_dq[-1]] >= nums[righ
t]: min_dq.pop()
            min_dq.append(right)
            while nums[max_dq[0]] - nums[min_dq[0]] > k:
                left += 1
                if max_dq[0] < left: max_dq.popleft()
                if min_dq[0] < left: min_dq.popleft()
            dp[right+1] = (prefix[right] - prefix[left] +
MOD) % MOD
            prefix[right+1] = (prefix[right] + dp[right+1]
) % MOD
        return dp[n]
```

PHASE 5 TEST & DEBUG

"Let me trace countPartitions(nums=[1,2,3,4],k=4)."

Count ways to partition nums into two non-empty parts where max of each part has difference <= k.

For each split point i (1<=i<=n-1): left=nums[0..i], right=nums[i+1..n-1].

max_left=max(left), max_right=max(right). Valid if |max_left-max_right|<=k.

Precompute prefix max and suffix max. Check each split in O(n).

[1,2,3,4], k=4: prefix_max=[1,2,3,4], suffix_max=[4,4,4,4].

Split at 1: max_left=1,max_right=4. |4-1|=3<=4 ok. Split at 2: |3-4|=1 ok. Split at 3: |4-4|=0 ok.

Return 3 ok

Monotonic stack=[prefix_max_idx,...] for tracking window max. stack=[0,1,2,...] evolves as prefix max computed. Pop when current suffix max makes window invalid. stack tracks partition boundaries.

Sliding window with monotonic deque=[suffix_max_idx,...]. popleft() removes expired front. deque tracks which partition is valid. deque=[front,...,back] for max-min constraint.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n)

Follow-up: count partitions where each part has sum >= k — same DP, different window condition.

Follow-up: Continuous Subarrays (#2762) — count subarrays not partitions.